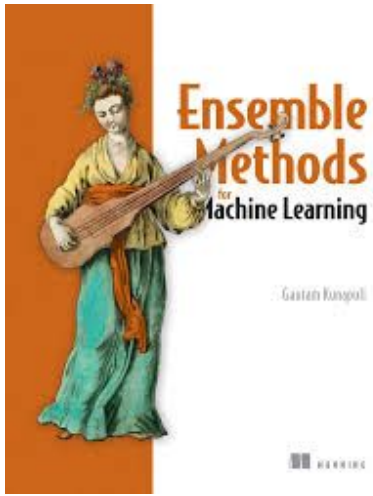
IF3270 Pembelajaran Mesin

Ensemble Methods

Tim Pengajar IF3270

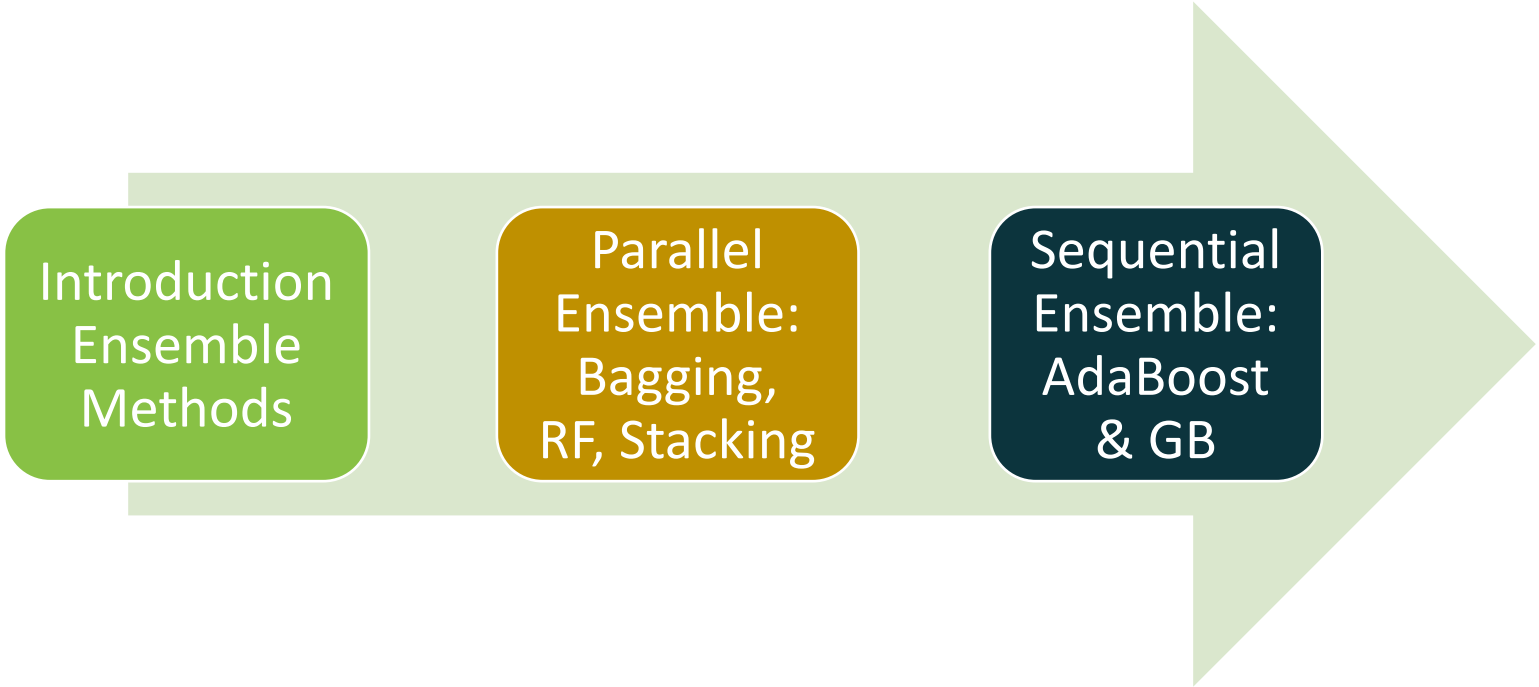
Reference



Kunapuli, G. (2023). *Ensemble methods for machine learning*. Simon and Schuster.

https://livebook.manning.com/book/ensemble-methods-for-machine-learning/chapter-1/19

Outline


Introduction Ensemble Methods
Parallel Ensemble: Bagging, RF, Stacking
Sequential Ensemble: AdaBoost & GB

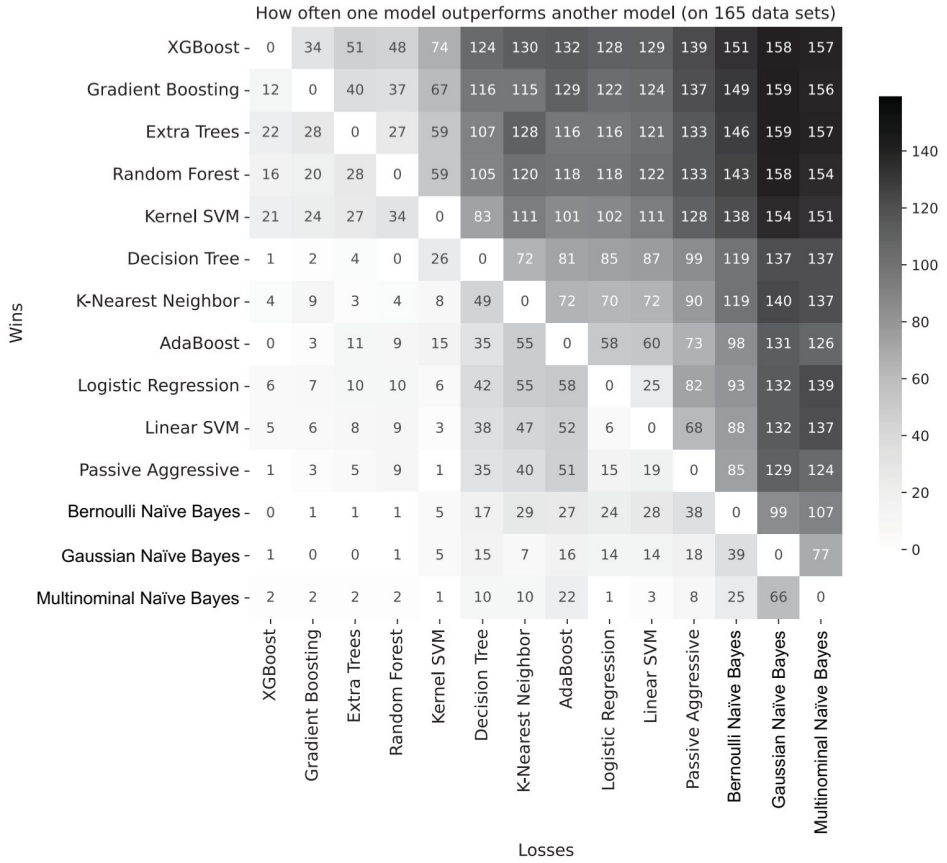
Why Ensemble Methods ?

- Ensemble methods have proven to be very successful on **structured data**.
    - Anthony Goldbloom, CEO of Kaggle, revealed in 2015 that the three most successful algorithms for structured problems were XGBoost, random forest, and gradient boosting, all ensemble methods.
- Ensemble methods have been a very effective tool to improve the performance of multiple existing models (Breiman, 2021)

https://livebook.manning.com/book/ensemble-methods-for-machine-learning/chapter-1/125

Why Ensemble Methods ?

- Olson et al. (2018) ranked machine-learning algorithm based on their performance on all benchmark data sets.
  - compared 13 popular ML algorithms from scikit-learn
  - PMLB (Penn Machine Learning Benchmarks)
- Ensemble methods outperformed other methods, that's why ensemble methods (specifically, tree-based ensembles) are considered state of the art for structured dataset.

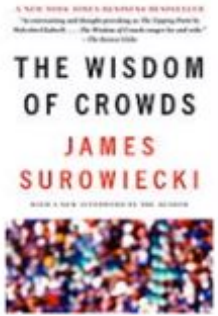


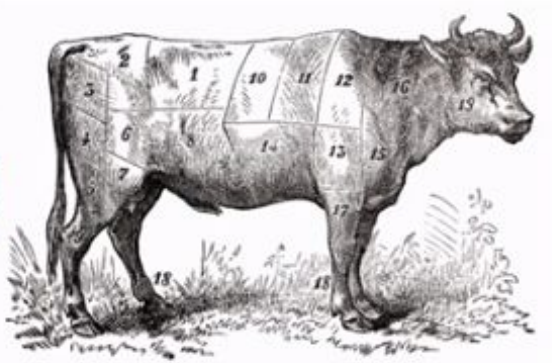
Olson, R. S., Cava, W. L., Mustahsan, Z., Varik, A., & Moore, J. H. (2018). Data-driven advice for applying machine learning to bioinformatics problems. In *Pacific symposium on biocomputing 2018: Proceedings of the pacific symposium* (pp. 192-203).

Romano, J. D., Le, T. T., La Cava, W., Gregg, J. T., Goldberg, D. J., Chakraborty, P., ... & Moore, J. H. (2022). PMLB v1. 0: an open-source dataset collection for benchmarking machine learning methods. *Bioinformatics*, *38*(3), 878-880.

What is Ensemble Methods ?

The Wisdom of Crowds





The wisdom of the crowd is the process of taking into account the **collective opinion** of a group of individuals rather than a **single expert** to answer a question.

average of 800 guesses = 1,197
actual weight of the ox = 1,198

https://vivifychangecatalyst.wordpress.com/2014/07/25/wisdom-of-crowd-crowdsourcing-performance-appraisal/

Ensemble Methods: The Wisdom of the Crowds

Dr. Randy Forrest works at a teaching hospital, and have a team with a diversity skills. Each resident has their own strengths.

Every time Dr. Forrest gets a new case, he solicits the opinions of his residents and then democratically selects the final diagnosis as **the most common one** from among all those proposed.

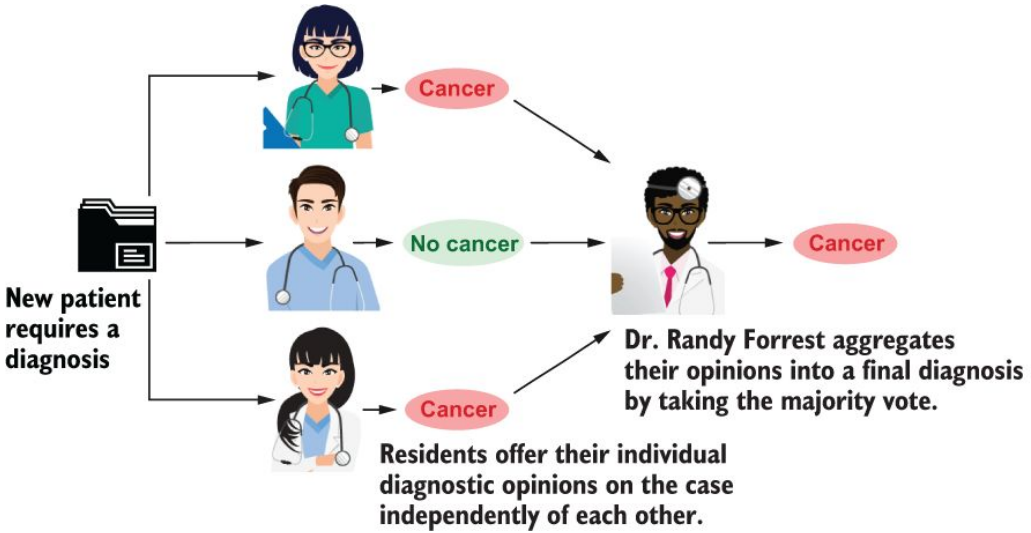
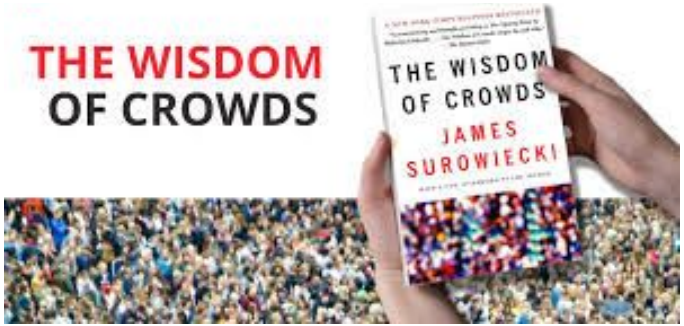


Figure 1.1 The diagnostic procedure followed by Dr. Randy Forrest every time he gets a new case is to ask all of his residents their opinions of the case. His residents offer their diagnoses: either the patient does or does not have cancer. Dr. Forrest then selects the majority answer as the final diagnosis put forth by his team.

Kunapuli, G. (2023). *Ensemble methods for machine learning*. Simon and Schuster.
https://livebook.manning.com/book/ensemble-methods-for-machine-learning/chapter-1/19

Ensemble Methods: The Wisdom of the Crowds

An ensemble method relies on the notion of “wisdom of the crowd”: the *combined* answer of many models is often better than any one individual answer.



The secrets to the success of ensemble methods are:

- Ensemble diversity (variety of opinions to choose from)
- Model aggregation into a single final decision

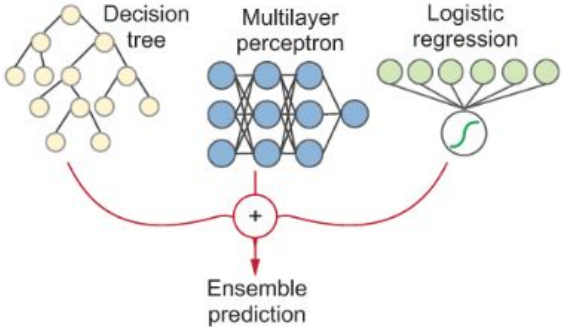
Diversity and independence are important because the best collective decisions are the product of disagreement and contest, not consensus or compromise.

https://livebook.manning.com/concept/machine-learning/parallel-ensemble-method

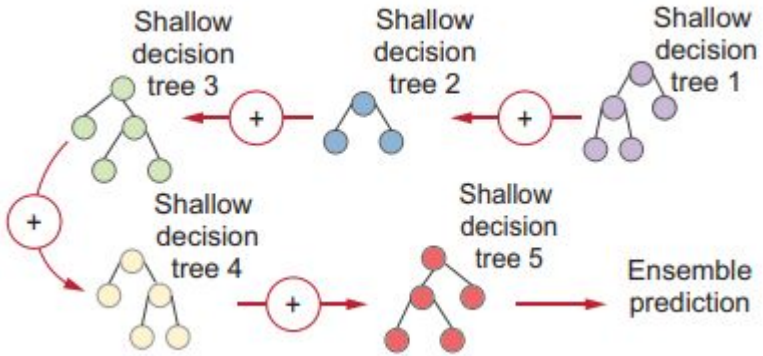
Terminology

- Base models/learners/estimators: individual machine-learning models
- Parallel vs sequential ensemble method: train each base model independently (or in parallel) vs sequentially in stages.
- Homogeneous vs heterogeneous ensembles: all the base learners are trained using the same vs different base-learning algorithm.

Heterogeneous parallel ensembles

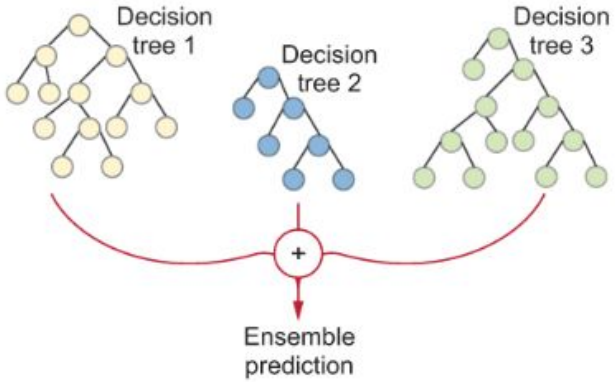


Sequential adaptive boosting ensembles

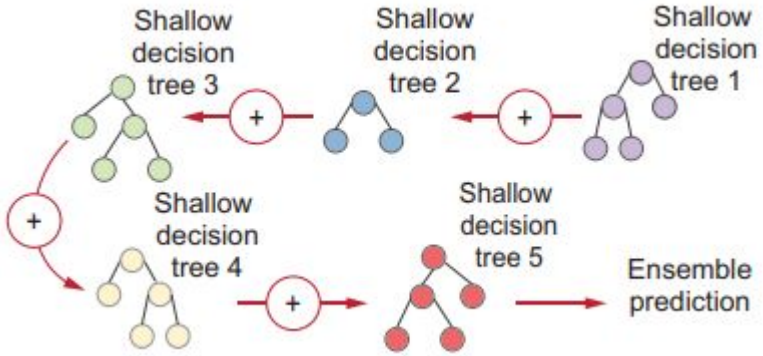


https://livebook.manning.com/book/ensemble-methods-for-machine-learning/chapter-1/125

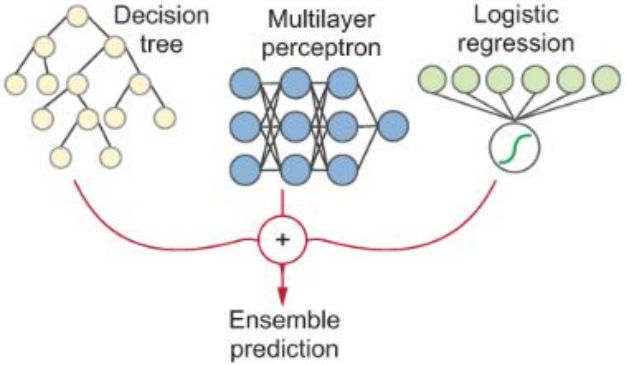
Homogeneous parallel ensembles



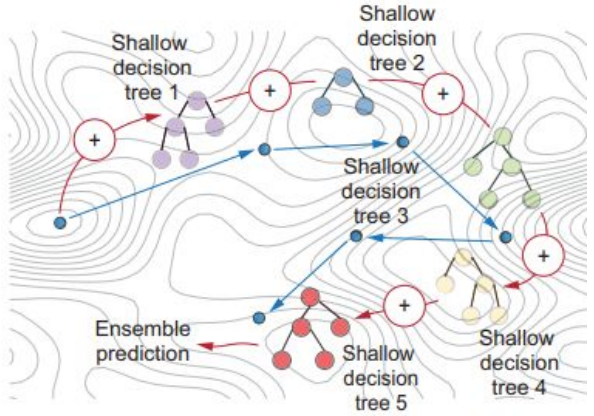
Sequential adaptive boosting ensembles



Heterogeneous parallel ensembles

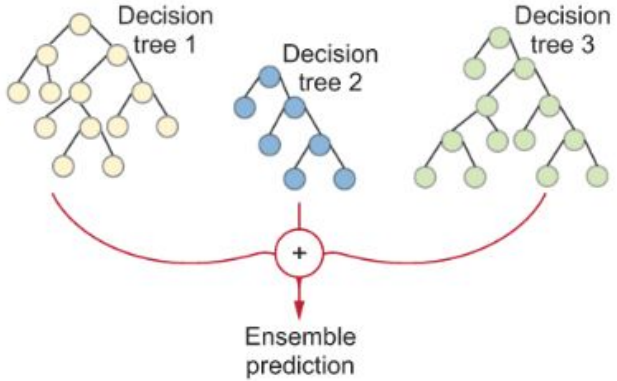


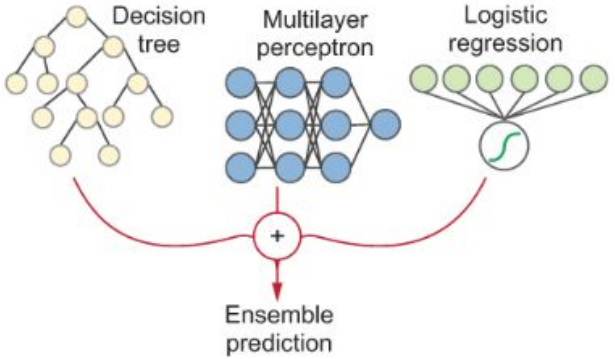
Sequential gradient boosting ensembles



Parallel Ensembles

exploit the *independence* of each base estimator



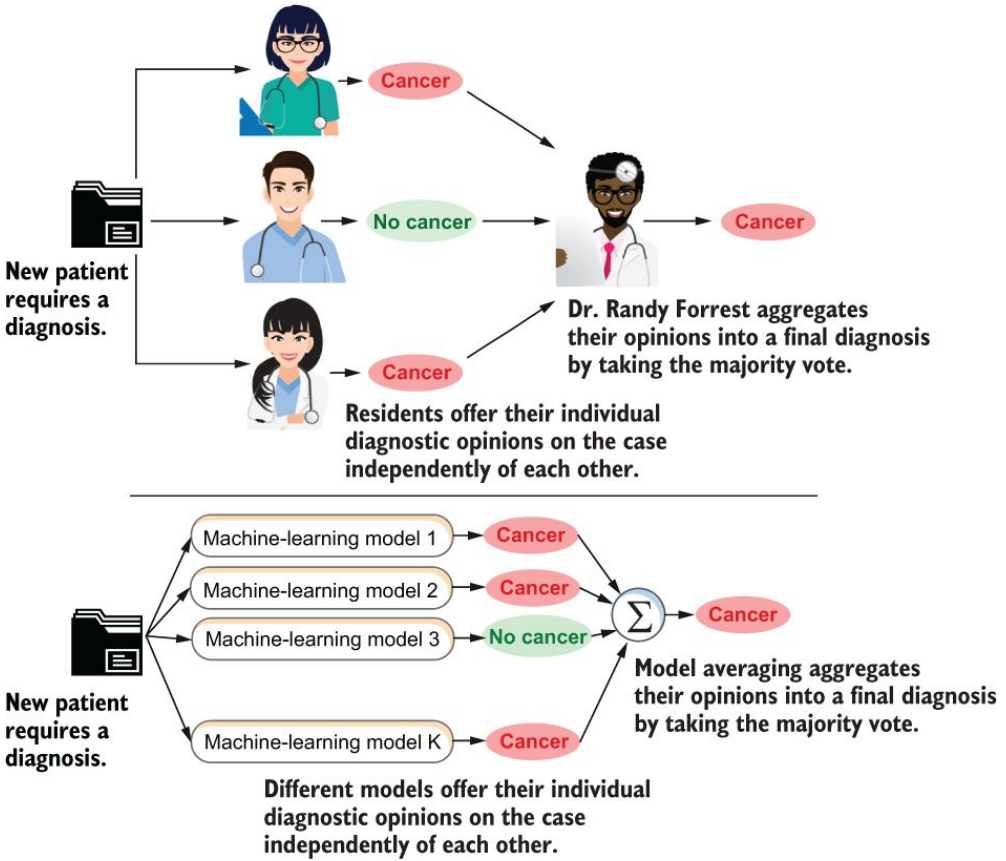


Homogeneous Parallel Ensembles

Homogeneous: component models are all trained using the same machine-learning algorithm

Parallel: train each component base estimator independently of the others (in parallel)

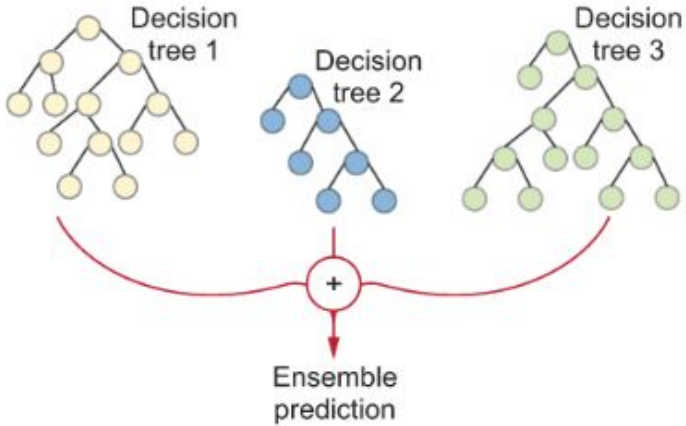
Dr. Randy Forrest's diagnostic process is an analogy of a parallel ensemble method



https://livebook.manning.com/concept/machine-learning/parallel-ensemble-method

Homogeneous Parallel Ensembles: Ensemble Diversity

We have a single data set, so how do we obtain different base models with the same learning algorithm in homogeneous ensemble ?

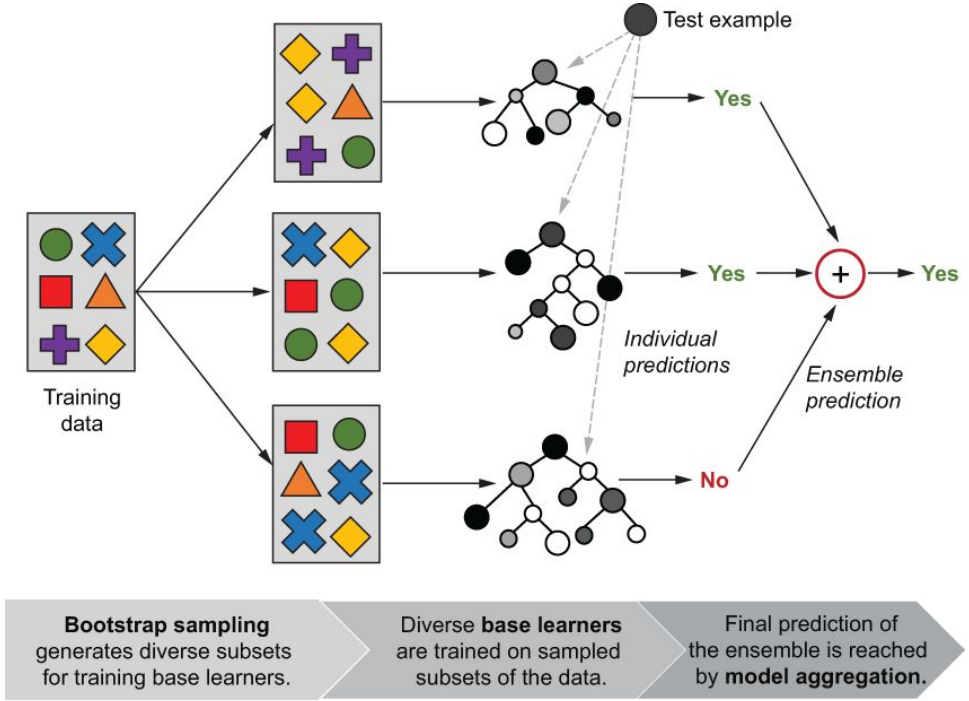


https://livebook.manning.com/concept/machine-learning/parallel-ensemble-method

Ensemble Diversity: Bagging - Bootstrap Aggregating

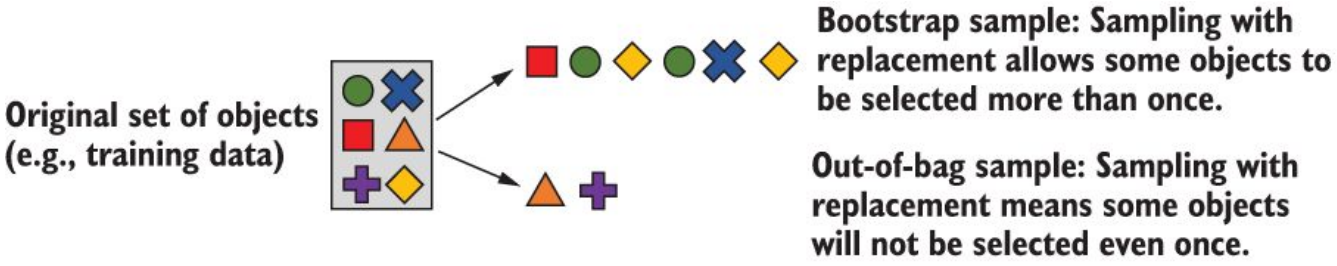
Training: bootstrap sampling (or sampling with replacement) is used to generate replicates of the training data set that are different from each other but drawn from the original data set. This ensures ensemble diversity.

Prediction: model aggregation is used to combine the predictions of the individual base learners into one ensemble prediction (classification: majority voting; regressing: simple averaging).



https://livebook.manning.com/concept/machine-learning/parallel-ensemble-method

Bootstrap Sampling

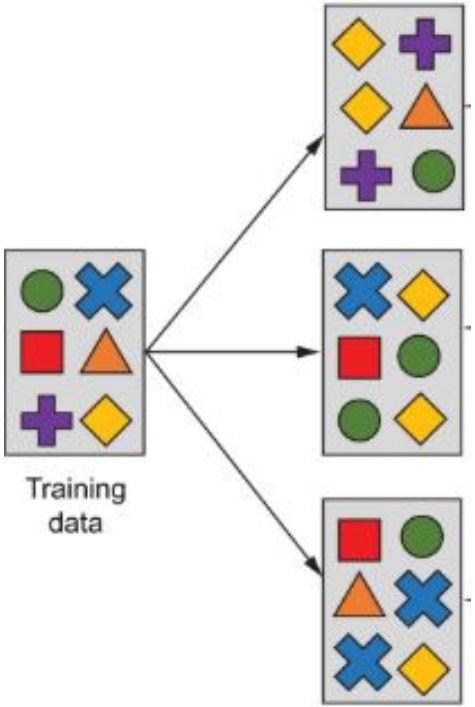



```
import numpy as np
bag = np.random.choice(range(0, 50), size=50, replace=True)
np.sort(bag)
```

```
array([ 1,  3,  4,  6,  7,  8,  9, 11, 12, 12, 14, 14, 15, 15, 21, 21, 21,
       24, 24, 25, 25, 26, 26, 29, 29, 31, 32, 32, 33, 33, 34, 34, 35, 35,
       37, 37, 39, 39, 40, 43, 43, 44, 46, 46, 48, 48, 48, 49, 49, 49])
```

```
oob = np.setdiff1d(range(0, 50), bag)
oob
```

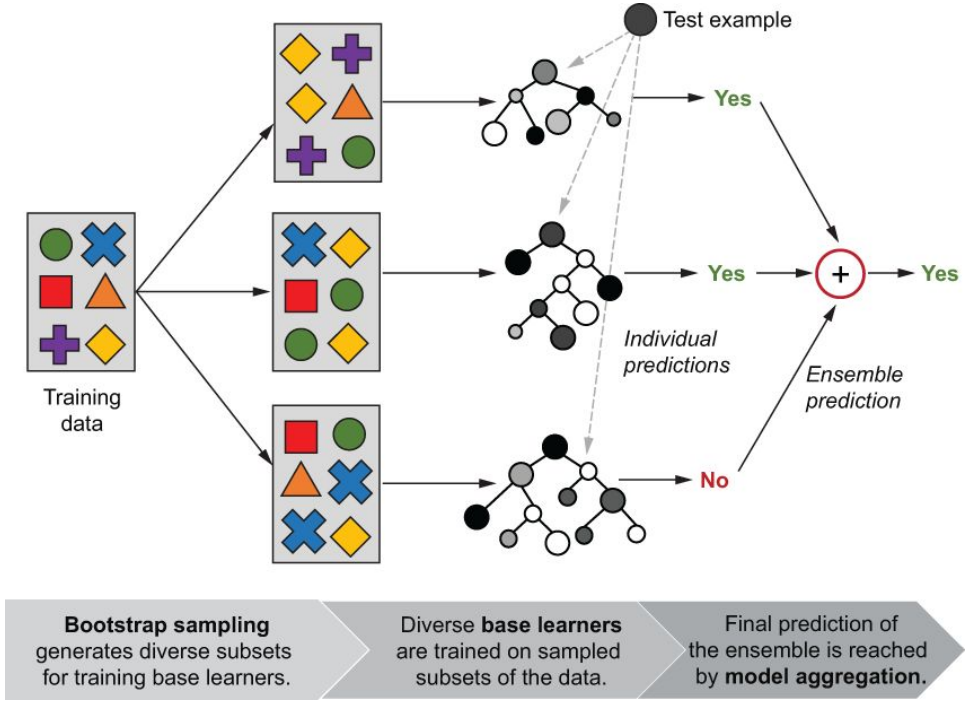
```
array([ 0,  2,  5, 10, 13, 16, 17, 18, 19, 20, 22, 23, 27, 28, 30, 36, 38,
       41, 42, 45, 47])
```



https://livebook.manning.com/concept/machine-learning/parallel-ensemble-method

Training with Bootstrap Sampling

Because different bootstrap samples will contain different examples repeating a different number of times, each base estimator will turn out to be somewhat different from the others.



https://livebook.manning.com/concept/machine-learning/parallel-ensemble-method

Training: Bagging with Decision Trees

```
import numpy as np
from sklearn.tree import DecisionTreeClassifier

rng = np.random.RandomState(seed=4190)
def bagging_fit(X, y, n_estimators, max_depth=5, max_samples=200):
    n_examples = len(y)
    estimators = [DecisionTreeClassifier(max_depth=max_depth)
                  for _ in range(n_estimators)]

    for tree in estimators:
        bag = np.random.choice(n_examples, max_samples,
                               replace=True)
        tree.fit(X[bag, :], y[bag])

    return estimators
```

Initializes a random seed

Creates a list of untrained base estimators

Generates a bootstrap sample

Fits a tree to the bootstrap sample

Prediction: Bagging with Decision Trees

```
from scipy.stats import mode

def bagging_predict(X, estimators):
    all_predictions = np.array([tree.predict(X)
                                for tree in estimators])
ypred, _ = mode(all_predictions, axis=0,
                keepdims=False)
    return np.squeeze(ypred)
```

Predicts each test example using each estimator in the ensemble

Makes the final predictions by majority voting

BaggingClassifier

BaggingClassifier

```
class sklearn.ensemble.BaggingClassifier(estimator=None, n_estimators=10, *, max_samples=1.0, max_features=1.0, bootstrap=True, bootstrap_features=False, oob_score=False, warm_start=False, n_jobs=None, random_state=None, verbose=0)
```
[source]

```
from sklearn.tree import DecisionTreeClassifier
clf = DecisionTreeClassifier()
clf.fit(Xtrn, ytrn)
ypred = clf.predict(Xtst)
print(accuracy_score(ytst, ypred))
```

```
0.9122807017543859
```

```
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import BaggingClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

dataset = load_breast_cancer()
X, y = dataset.data, dataset.target
Xtrn, Xtst, ytrn, ytst = train_test_split(X, y, test_size=0.1, random_state=42)
clf = BaggingClassifier(estimator=DecisionTreeClassifier(max_depth=3), max_features=1,
oob_score=True, random_state=42)
clf.fit(Xtrn, ytrn)
ypred = clf.predict(Xtst)
print(accuracy_score(ytst, ypred))
```

```
0.9824561403508771
```

```
clf.oob_score_
```

```
0.857421875
```

Random Forest

- an ensemble of randomized decision trees constructed using a special extension of bagging.
- Problem with bagging: the same small number of dominant features are repeatedly used in different trees. This makes the ensemble less diverse.
  - Solution: randomly samples features before creating a decision node
  - **Randomized decision trees** are trained using a modified decision-tree learning algorithm
- This additional source of randomness increases ensemble diversity and generally leads to better predictive performance.

https://livebook.manning.com/concept/machine-learning/parallel-ensemble-method

Randomized Decision Tree

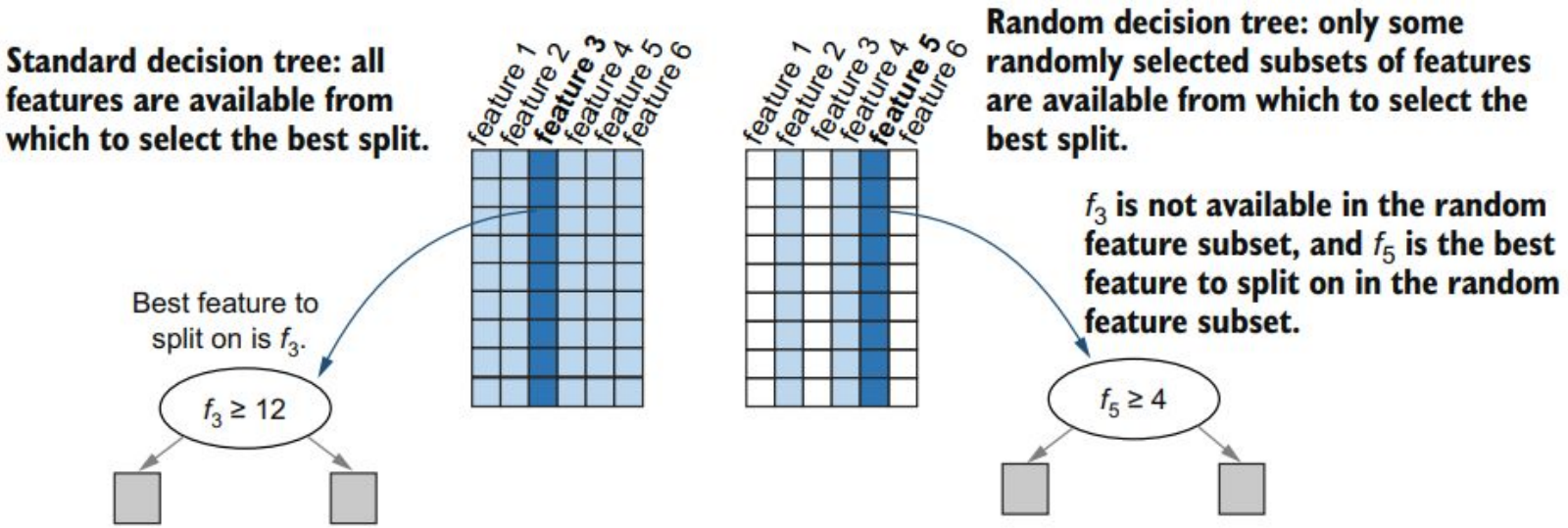


Figure 2.7 Random forests use a modified tree learning algorithm, where a random feature subset is first chosen before the best splitting criterion for each decision node is identified. The unshaded columns represent features that have been left out; the lightly shaded columns represent available features from which the best feature is chosen, shown in the darkly shaded columns.

Bootstrap Sampling + Randomized tree = Random Forest

When randomized tree learning (with a random sampling of features) is combined with bootstrap sampling (with a random sampling of training examples), we obtain an ensemble of randomized decision trees, known as a random decision forest or simply random forest.

Algorithm 1: Pseudo code for the random forest algorithm

To generate c classifiers:
for i = 1 to c **do**
  Randomly sample the training data D with replacement to produce D_i
  Create a root node, N_i containing D_i
  Call BuildTree(N_i)
end for

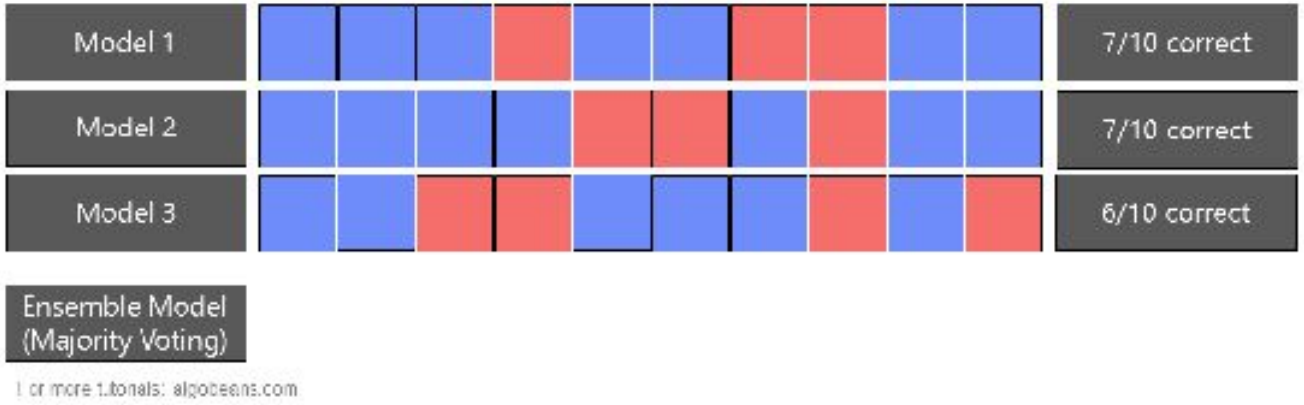
BuildTree(N):
if N contains instances of only one class **then**
  return
else
  Randomly select x% of the possible splitting features in N
  Select the feature F with the highest information gain to split on
  Create f child nodes of N, N_1,..., N_f, where F has f possible values (F_1, ... , F_f)
  for i = 1 to f **do**
    Set the contents of N_i to D_i, where D_i is all instances in N that match F_i
    Call BuildTree(N_i)
  end for
end if

https://scikit-learn.org/stable/modules/ensemble.html#forest

- Each tree in the ensemble is built from a sample drawn with replacement (i.e., a bootstrap sample) from the training set.
- When splitting each node during the construction of a tree, the best split is found either from all input features or a random subset of size `max_features`.
- the scikit-learn implementation combines classifiers by **averaging their probabilistic prediction,** instead of letting each classifier vote for a single class.

https://www.researchgate.net/publication/335375894_Forecasting_mining_capital_cost_for_open-pit_mining_projects_based_on_artificial_neural_network_approach/figures?lo=1

Majority Voting



The predicted class probabilities of an input sample are computed as the mean predicted class probabilities of the trees in the forest.

https://www.kdnuggets.com/2016/09/reandom-forest-criminal-tutorial.html
https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html#sklearn.ensemble.RandomForestClassifier.predict_proba

RandomForest

RandomForestClassifier

class sklearn.ensemble.**RandomForestClassifier**(*n_estimators=100*, ***, *criterion='gini'*, *max_depth=None*, *min_samples_split=2*, *min_samples_leaf=1*, *min_weight_fraction_leaf=0.0*, *max_features='sqrt'*, *max_leaf_nodes=None*, *min_impurity_decrease=0.0*, *bootstrap=True*, *oob_score=False*, *n_jobs=None*, *random_state=None*, *verbose=0*, *warm_start=False*, *class_weight=None*, *ccp_alpha=0.0*, *max_samples=None*, *monotonic_cst=None*) [source]

```
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

dataset = load_breast_cancer()
X, y = dataset.data, dataset.target
Xtrn, Xtst, ytrn, ytst = train_test_split(X, y, test_size=0.1, random_state=42)
clf = RandomForestClassifier(max_features=10, max_depth=5, oob_score=True, random_state=42)
clf.fit(Xtrn, ytrn)
ypred = clf.predict(Xtst)
print(accuracy_score(ytst, ypred))
```

```
0.9649122807017544
```

```
clf.oob_score_
```

```
0.953125
```

Why Random Forest ?

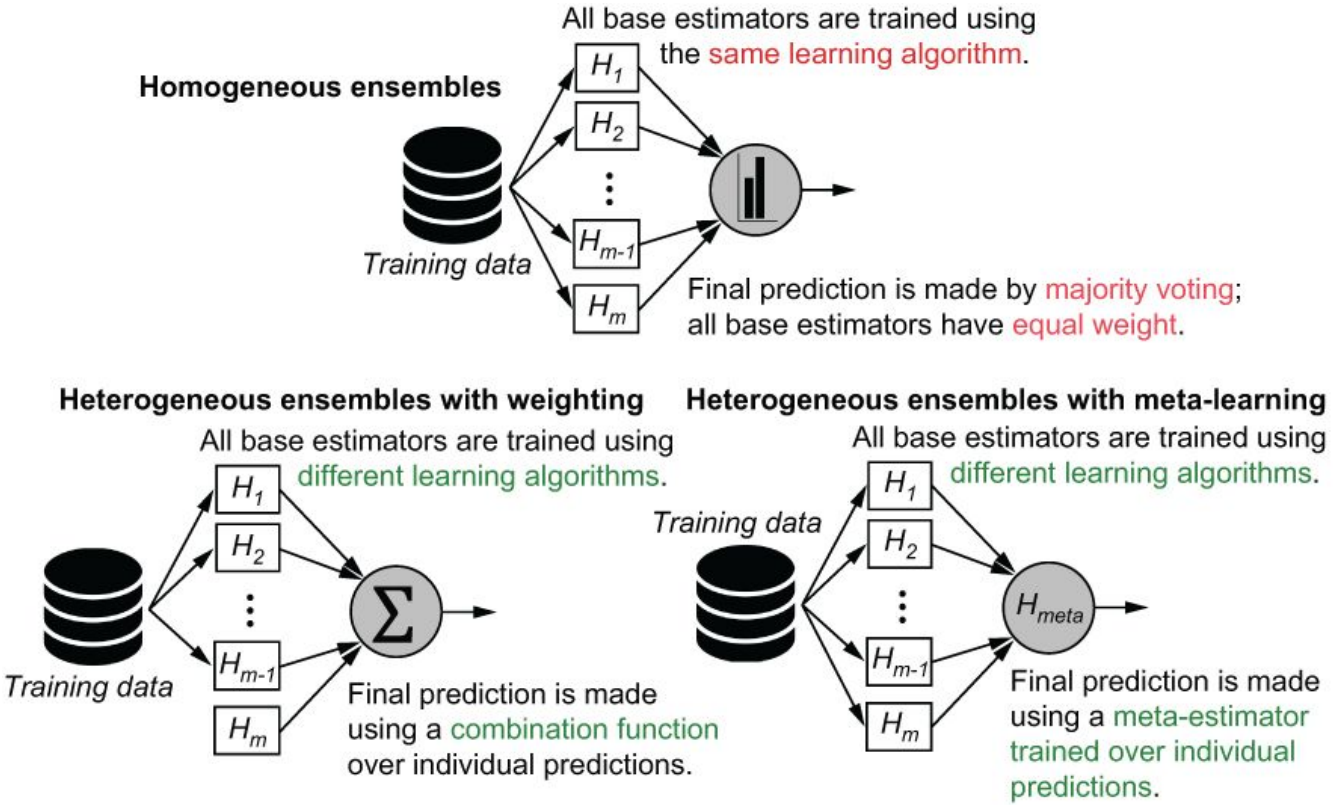
- Random forest is also well known as one of the most accurate and as a fast learning method independently from the nature of the datasets, as shown by various recent comparative studies (Fernández-Delgado et al. 2014; Caruana and Niculescu-Mizil 2006; Rokach 2016).
- Ensemble learning algorithms like random forests are well suited for medium to large datasets.

Random Forest: d >>n

- When the number of independent variables (d) is larger than the number of observations (n), linear regression and logistic regression algorithms will not run, because the number of parameters to be estimated exceeds the number of observations. Random forest works because not all predictor variables are used at once.

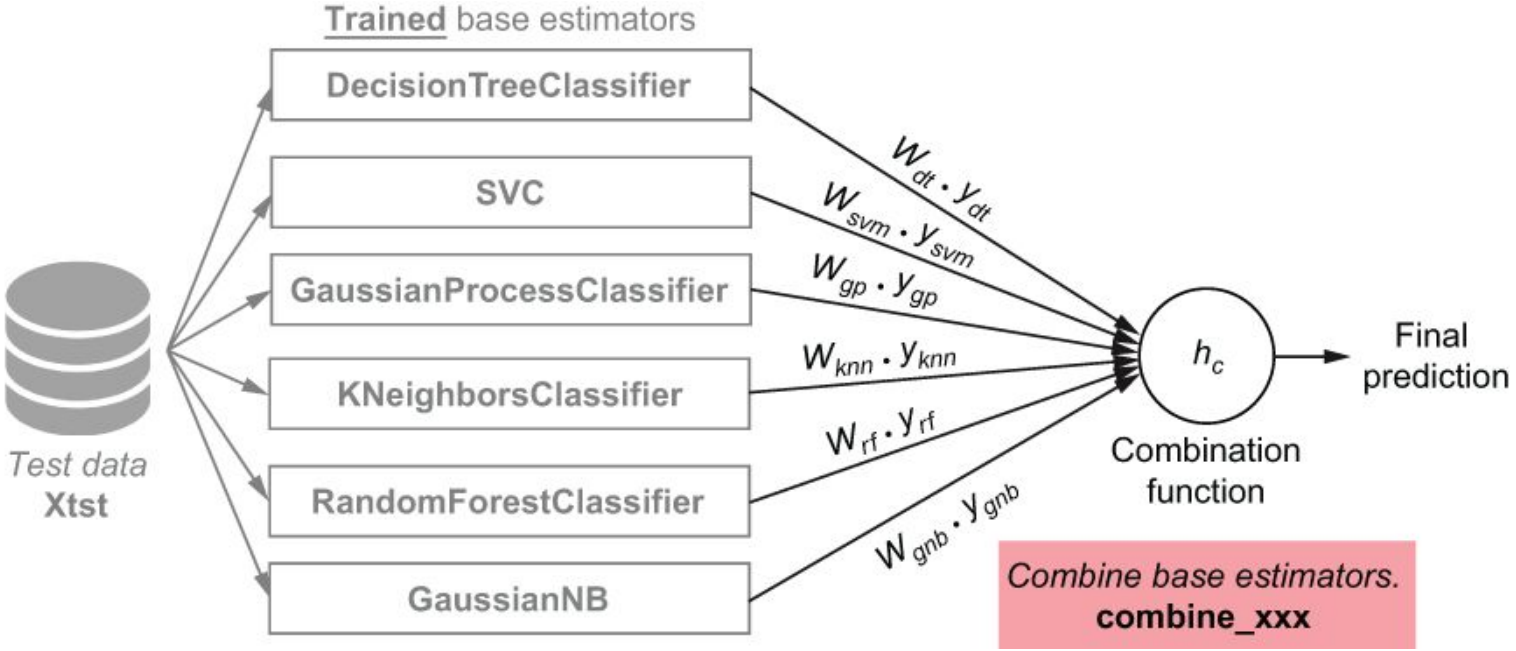
  Schonlau, M., & Zou, R. Y. (2020). The random forest algorithm for statistical learning. *The Stata Journal*, *20*(1), 3-29.

Heterogeneous Parallel Ensembles



https://livebook.manning.com/concept/machine-learning/parallel-ensemble-method

Heterogeneous Ensembles with Weighting



$$y_{final} = w_1 \cdot y_1 + w_2 \cdot y_2 + \cdots + w_m \cdot y_m = \sum_{t=1}^{m} w_t \cdot y_t$$

https://livebook.manning.com/concept/machine-learning/parallel-ensemble-method

Heterogeneous Ensembles with Weighting: Accuracy Weighting

Base classifier 1: 3nn Classifier: validation accuracy = 0.9646

Base classifier 2: NB classifier: validation accuracy = 0.885

Weights:

W3nn = 0.9646 / (0.966+0.885) = 0.522

Wnb = 0.885 / (0.966+0.885) = 0.478

$$y = \underbrace{\frac{\alpha_{3nn}}{\alpha_{3nn} + \alpha_{gnb}}}_{w_{3nn}} \cdot y_{3nn} + \underbrace{\frac{\alpha_{gnb}}{\alpha_{3nn} + \alpha_{gnb}}}_{w_{gnb}} \cdot y_{gnb}$$

Heterogeneous Ensembles with Weighting: Prediction

```
def combine_using_accuracy_weighting(X, estimators,
                                     Xval, yval):
    n_estimators = len(estimators)
    yval_individual = predict_individual(Xval,
                          estimators, proba=False)

    wts = [accuracy_score(yval, yval_individual[:, i])
           for i in range(n_estimators)]

wts /= np.sum(wts)

ypred_individual = predict_individual(X, estimators, proba=False)
y_final = np.dot(ypred_individual, wts)

return np.round(y_final)
```

Takes the validation set as input

Gets individual predictions on the validation set

Sets the weight for each base classifier as its accuracy score

Normalizes the weights

Computes the weighted combination of individual labels efficiently

Converts the combined prediction into a 0–1 label by rounding

Heterogeneous Ensembles with Weighting

Task example: binary classification (class: 0, 1)

Base classifier 1 (clf1): 3nn Classifier → w3nn = 0.522

Base classifier 2 (clf2): NB classifier → wnb = 0.478

New data: x

predict_individual(**x**, clf, proba=False)

clf1.predict(**x**) = 1;

clf2.predict(**x**) = 0;

Voting.predict(**x**, hard) = round(0.522*1+0.478*0) = 1

predict_individual(**x**, clf, proba=True)

clf1.predict_proba(**x**) = [0.4, 0.6] # prob_class_0, prob_class_1

clf2.predict_proba(**x**) = [0.6, 0.4]

Voting.predict(**x**, soft) = {0.522*0.4+0.478*0.6, 0.522*0.6+0.478*0.4} = {0.4956, 0.5044}

Voting: Example

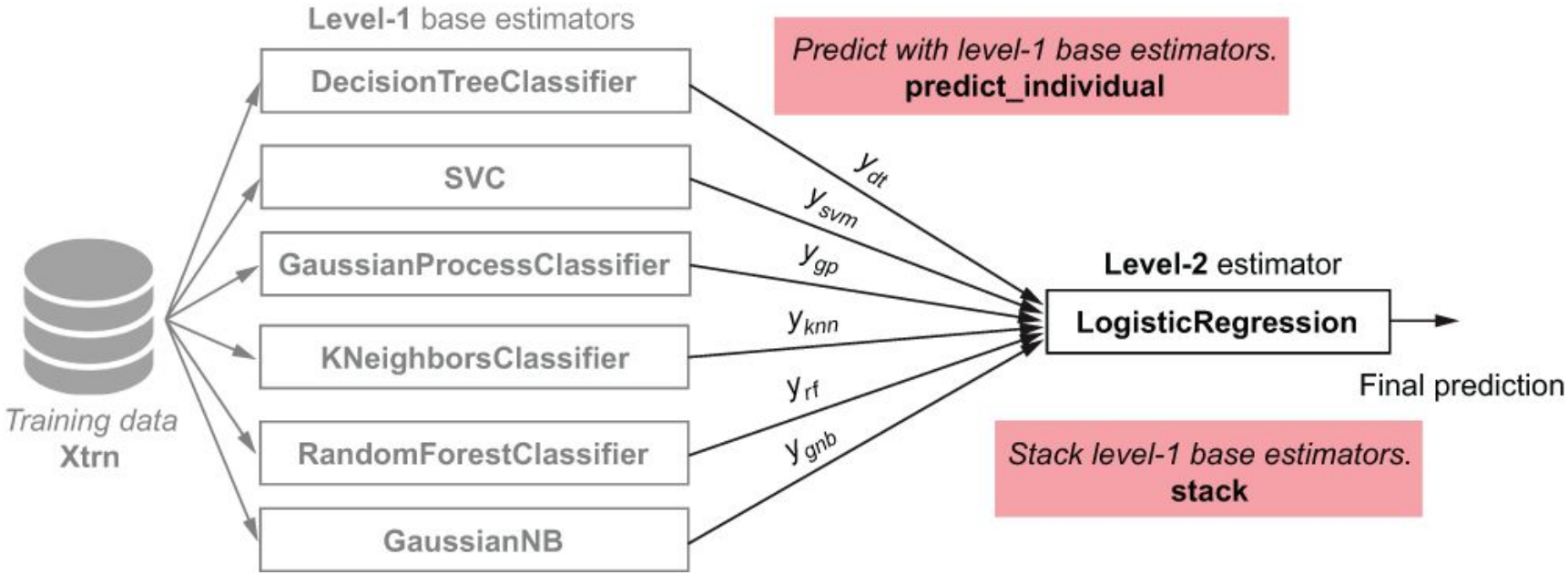
VotingClassifier

```
class sklearn.ensemble.VotingClassifier(estimators, *, voting='hard',
weights=None, n_jobs=None, flatten_transform=True,
verbose=False)                                                    [source]
```

```
>>> import numpy as np
>>> from sklearn.linear_model import LogisticRegression
>>> from sklearn.naive_bayes import GaussianNB
>>> from sklearn.ensemble import RandomForestClassifier, VotingClassifier
>>> clf1 = LogisticRegression(random_state=1)
>>> clf2 = RandomForestClassifier(n_estimators=50, random_state=1)
>>> clf3 = GaussianNB()
>>> X = np.array([[-1, -1], [-2, -1], [-3, -2], [1, 1], [2, 1], [3, 2]])
>>> y = np.array([1, 1, 1, 2, 2, 2])
>>> eclf1 = VotingClassifier(estimators=[
...         ('lr', clf1), ('rf', clf2), ('gnb', clf3)], voting='hard')
>>> eclf1 = eclf1.fit(X, y)
>>> print(eclf1.predict(X))
[1 1 1 2 2 2]
```

```
>>> eclf2 = VotingClassifier(estimators=[
...         ('lr', clf1), ('rf', clf2), ('gnb', clf3)],
...         voting='soft')
```

Heterogeneous Ensembles with Meta Learning



https://livebook.manning.com/concept/machine-learning/parallel-ensemble-method

Stacking: Example

StackingClassifier

```
class sklearn.ensemble.StackingClassifier(estimators,
final_estimator=None, *, cv=None, stack_method='auto', n_jobs=None,
passthrough=False, verbose=0)                                    [source]
```

The `estimators` parameter corresponds to the list of the estimators which are stacked together in parallel on the input data. It should be given as a list of names and estimators:

```
>>> from sklearn.linear_model import RidgeCV, LassoCV
>>> from sklearn.neighbors import KNeighborsRegressor
>>> estimators = [('ridge', RidgeCV()),
...               ('lasso', LassoCV(random_state=42)),
...               ('knr', KNeighborsRegressor(n_neighbors=20,
...                                           metric='euclidean'))]
```

The `final_estimator` will use the predictions of the `estimators` as input. It needs to be a classifier or a regressor when using **`StackingClassifier`** or **`StackingRegressor`**, respectively:

```
>>> from sklearn.ensemble import GradientBoostingRegressor
>>> from sklearn.ensemble import StackingRegressor
>>> final_estimator = GradientBoostingRegressor(
...     n_estimators=25, subsample=0.5, min_samples_leaf=25, max_features=1,
...     random_state=42)
>>> reg = StackingRegressor(
...     estimators=estimators,
...     final_estimator=final_estimator)
```

Stacking: Multi layers

Multiple stacking layers can be achieved by assigning `final_estimator` to a `StackingClassifier` or `StackingRegressor`:

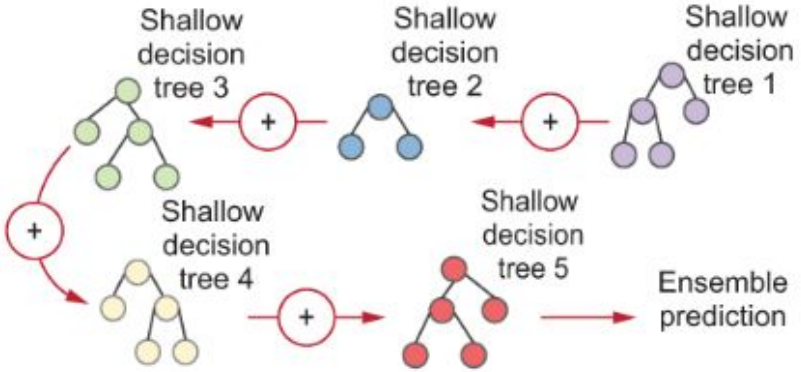
```
>>> final_layer_rfr = RandomForestRegressor(
...     n_estimators=10, max_features=1, max_leaf_nodes=5,random_state=42)
>>> final_layer_gbr = GradientBoostingRegressor(
...     n_estimators=10, max_features=1, max_leaf_nodes=5,random_state=42)
>>> final_layer = StackingRegressor(
...     estimators=[('rf', final_layer_rfr),
...                 ('gbrt', final_layer_gbr)],
...     final_estimator=RidgeCV()
...     )
>>> multi_layer_regressor = StackingRegressor(
...     estimators=[('ridge', RidgeCV()),
...                 ('lasso', LassoCV(random_state=42)),
...                 ('knr', KNeighborsRegressor(n_neighbors=20,
...                                             metric='euclidean'))],
...     final_estimator=final_layer
... )
>>> multi_layer_regressor.fit(X_train, y_train)
StackingRegressor(...)
>>> print('R2 score: {:.2f}'
...       .format(multi_layer_regressor.score(X_test, y_test)))
R2 score: 0.53
```

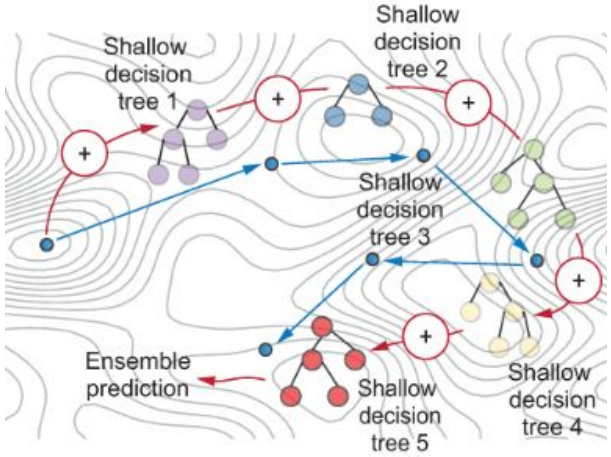
Overview Ensemble Methods

- Wisdom of the crowds: the *combined* answer of many models is often better than any one individual answer.
  - Ensemble diversity → training
  - Model aggregation into a single final decision → inference
- Parallel Ensembles:
  - Strong learners
  - Homogeneous parallel ensembles
    - Bagging: bootstrap sampling + aggregating
    - Random Forest: bootstrap sampling + random subspace + aggregating
  - Heterogeneous parallel ensembles
    - Heterogeneous ensembles with weighting: heterogen learning algorithm + weighting agg
    - Heterogeneous ensembles with meta learning: heterogen learning algorithm + stacking

Sequential Ensembles

exploit the *dependence* of base estimators





Parallel vs Sequential Ensembles

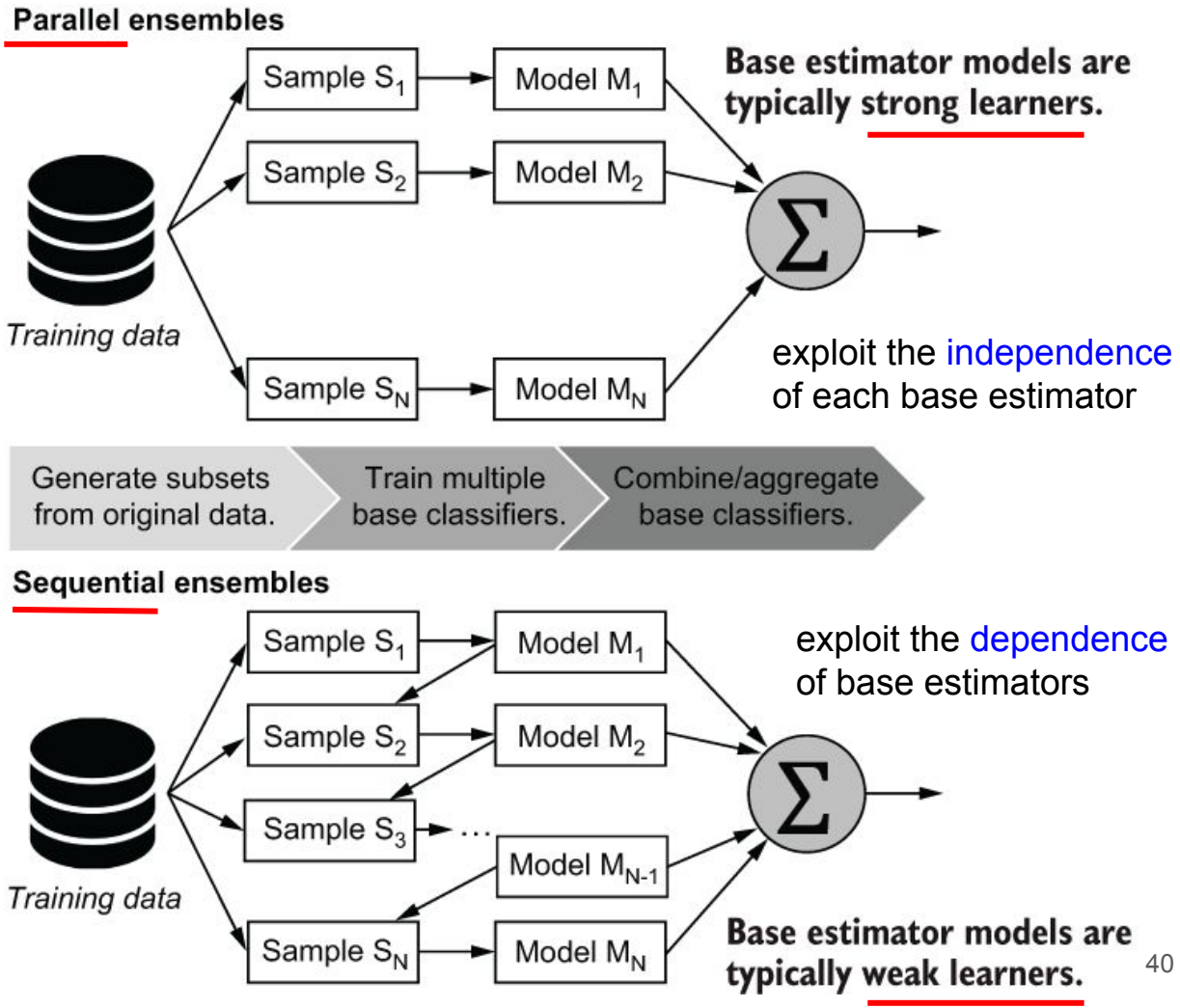
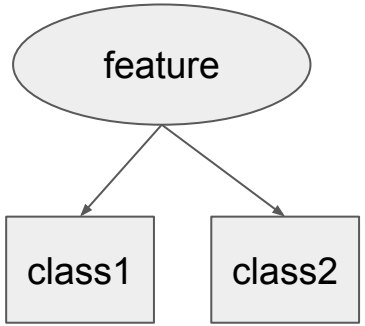


Figure 4.1 Differences between parallel and sequential ensembles: (1) base estimators in parallel ensembles are trained independently of each other, while in sequential ensembles, they are trained to improve on the predictions of the previous base estimator; (2) sequential ensembles typically use weak learners as base estimators.

Boosting

- Sequential ensemble methods such as boosting aim to combine several weak learners into a single strong learner.
  - Boosting literally aims to boost the performance of a collection of weak learners (very simple model that has performance slightly better than random chance for binary classification)
- Decision trees are often used as base estimators for sequential ensembles. Boosting algorithms typically use decision stumps, or decision trees of depth 1
- Methods: AdaBoost, Gradient Boosting

Decision stumps



Boosting Classifier

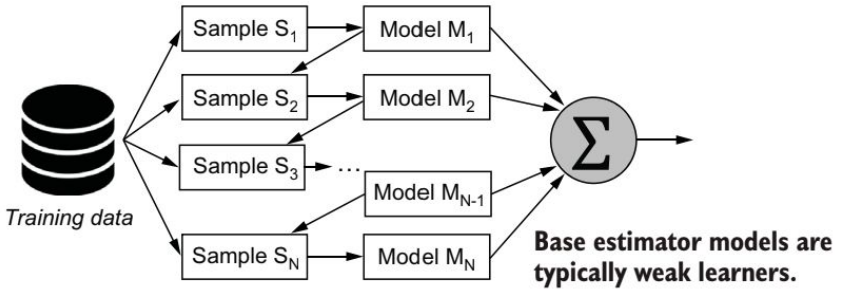
```
class sklearn.ensemble.AdaBoostClassifier(estimator=None, *, n_estimators=50,
learning_rate=1.0, random_state=None)

class sklearn.ensemble.GradientBoostingClassifier(*, loss='log_loss',
learning_rate=0.1, n_estimators=100, subsample=1.0, criterion='friedman_mse',
min_samples_split=2, min_samples_leaf=1, min_weight_fraction_leaf=0.0, max_depth=3,
min_impurity_decrease=0.0, init=None, random_state=None, max_features=None,
verbose=0, max_leaf_nodes=None, warm_start=False, validation_fraction=0.1,
n_iter_no_change=None, tol=0.0001, ccp_alpha=0.0)
```

```
class lightgbm.LGBMClassifier(*, boosting_type='gbdt', num_leaves=31, max_depth=-1,
learning_rate=0.1, n_estimators=100, subsample_for_bin=200000, objective=None, class_weight=None,
min_split_gain=0.0, min_child_weight=0.001, min_child_samples=20, subsample=1.0, subsample_freq=0,
colsample_bytree=1.0, reg_alpha=0.0, reg_lambda=0.0, random_state=None, n_jobs=None,
importance_type='split', **kwargs)
```

```
CatBoostClassifier(iterations=2, depth=2, learning_rate=1, loss_function='Logloss',
verbose=True)
```

Adaptive Boosting (AdaBoost)

AdaBoost is an adaptive algorithm: at every iteration, it trains a new base estimator h_i that fixes the mistakes made by the previous base estimator h_{i-1} → prioritizes misclassified training examples.



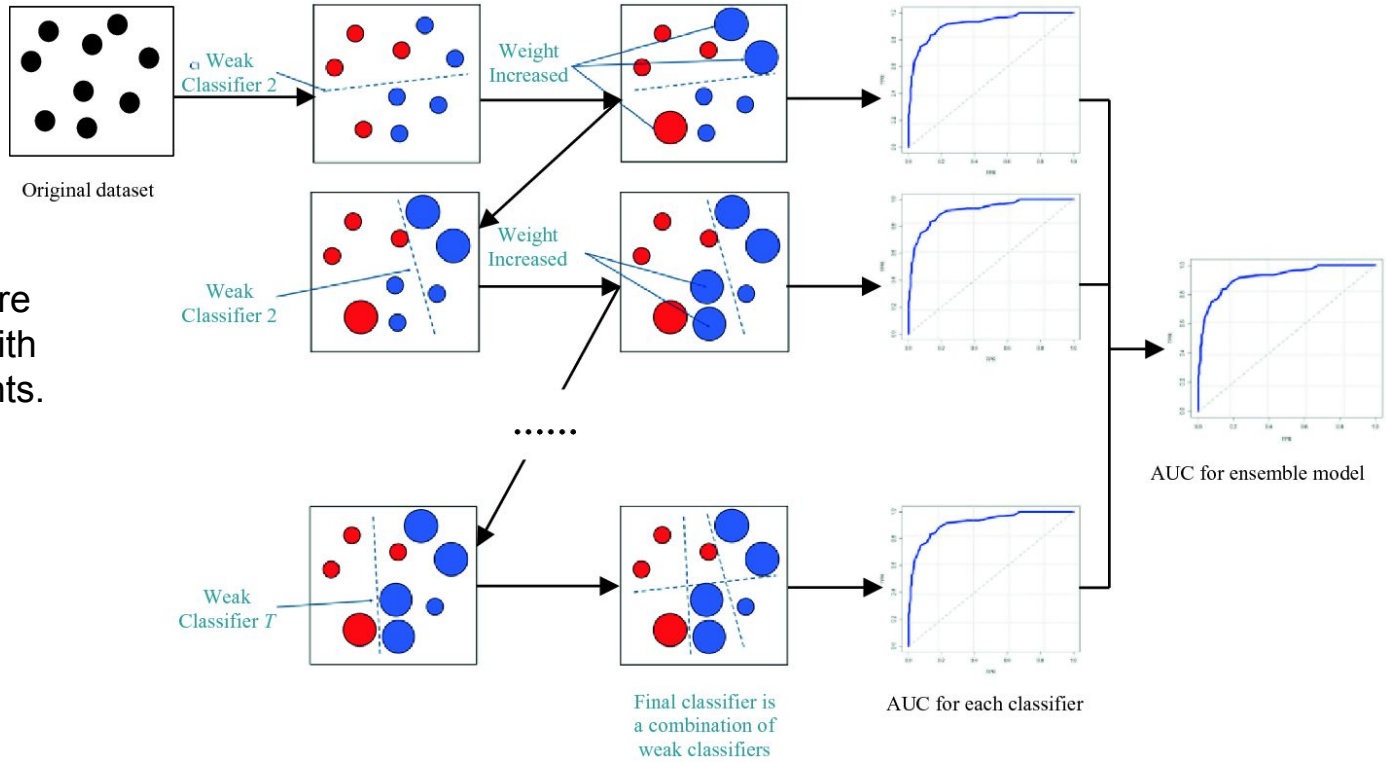
AdaBoost does this by maintaining weights over individual training examples (weights reflect the relative importance of training examples).

The final boosted classifier, H, combines the votes of each individual classifier, where the weight of each classifier's vote is a function of its accuracy. This forms a weighted linear combination of base estimators.

Kunapuli, G. (2023). *Ensemble methods for machine learning*. Simon and Schuster.
Han, J., Pei, J., & Tong, H. (2022). *Data mining: concepts and techniques*. Morgan kaufmann.

Adaptive Boosting (AdaBoost): Illustration

Initially, all examples are initialized with equal weights.



https://datascience.eu/machine-learning/gradient-boosting-what-you-need-to-know/

AdaBoost: Training (Kunapuli, 2023)

Kunapuli, G. (2023). *Ensemble methods for machine learning*. Simon and Schuster.

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
import numpy as np

def fit_boosting(X, y, n_estimators=10):
    n_samples, n_features = X.shape
    D = np.ones((n_samples, ))
    estimators = []

    for t in range(n_estimators):
        D = D / np.sum(D)

        h = DecisionTreeClassifier(max_depth=1)
        h.fit(X, y, sample_weight=D)

        ypred = h.predict(X)
        e = 1 - accuracy_score(y, ypred,
                               sample_weight=D)
        a = 0.5 * np.log((1 - e) / e)

        m = (y == ypred) * 1 + (y != ypred) * -1
        D *= np.exp(-a * m)

        estimators.append((a, h))

    return estimators
```

Initializes an empty ensemble

Nonnegative weights, initialized to 1

Normalizes the weights so they sum to 1

Trains a weak learner (h_t) with weighted examples

Computes the training error (ε_t) and the weight (α_t) of the weak learner

Updates the example weights: increase for misclassified examples, decrease for correctly classified examples

Saves the weak learner and its weight

1) Train a weak learner $h_t(\mathbf{x})$ using weighted dataset <xi,yi,Di>*

2) Compute training error e_t dari $h_t(\mathbf{x})$

3) Computer model weight α_t

$$\alpha_t = \frac{1}{2}\log\left(\frac{1-\epsilon_t}{\epsilon_t}\right)$$

4) Update weight

$$D_i^{t+1} = D_i^t \cdot \begin{cases} e^{\alpha_t}, \text{ if misclassified,} \\ e^{-\alpha_t}, \text{ if correctly classified} \end{cases}$$

Weak Learner Weights

t=i: training accuracy = 0.9 → error = 0.1

α_i= 0.5*ln((1-0.1)/0.1)=1.099

t=i+1: training accuracy = 0.5 → error = 0.5

α_{i+1}= 0.5*ln((1-0.5)/0.5)= 0

t=i+2: training accuracy = 0.1 → error = 0.9

α_{i+2}= 0.5*ln((1-0.9)/0.9)= -1.099

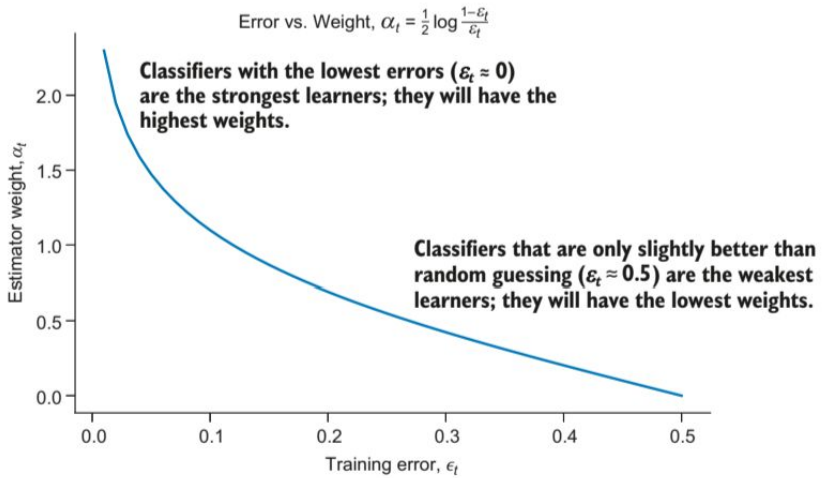


Figure 4.7 AdaBoost assigns stronger learners (which have lower training errors) higher weights, and assigns weaker learners (which have higher training errors) lower weights.

Instance Weights in Adaptive Boosting: Illustration

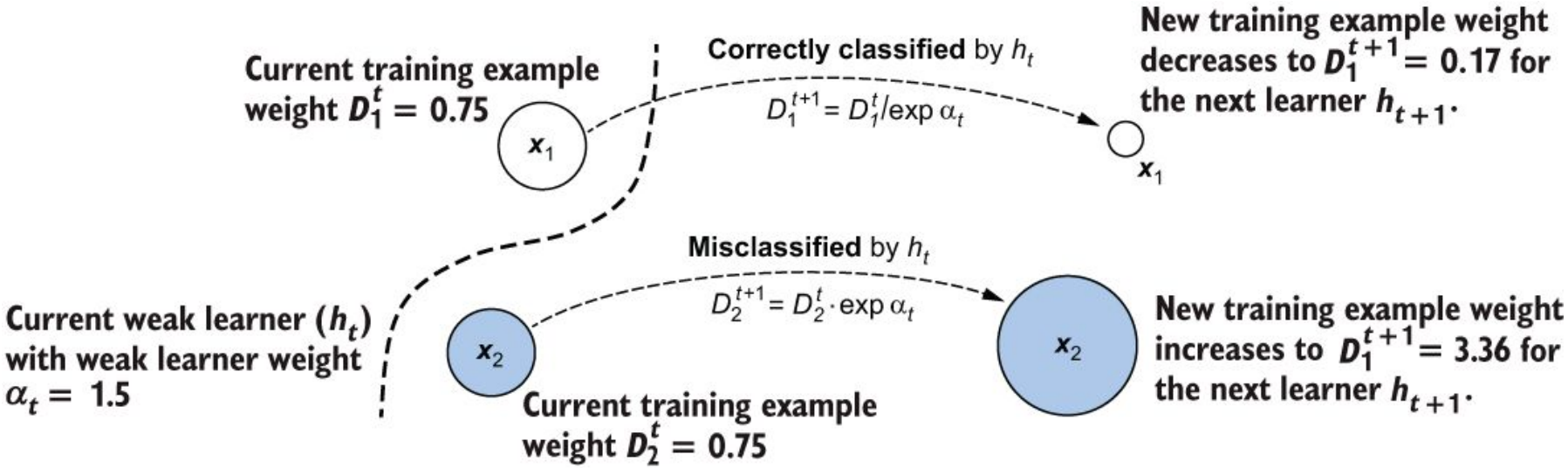


Figure 4.8 In iteration t, two training examples, x_1 and x_2, have the same weights. x_1 is correctly classified, while x_2 is misclassified by the current base estimator h_t. As the goal in the next iteration is to learn a classifier h_{t+1} that fixes the mistakes of h_t, AdaBoost increases the weight of the misclassified example x_2, while decreasing the weight of the correctly classified example x_1.This allows the base-learning algorithm to prioritize x_2 during training in iteration t+1.

Instance Weights in Adaptive Boosting: Illustration

- Notation:
  x_i: instance i
  D_i: weight of instance i
  $c(x_i)$: label of x_i

- Iteration t:
  $D_1^t = D_2^t = 0.75$
  Update α_t: $\alpha_t = 1.5$

$$D_i^{t+1} = D_i^t \cdot \begin{cases} e^{\alpha_t}, \text{ if misclassified,} \\ e^{-\alpha_t}, \text{ if correctly classified} \end{cases}$$

Update instance weights:
$h_t(x_1) = c(x_1)$ #correctly classified $\rightarrow D_1^{t+1} = D_1^t / e^{\alpha t} = 0.75/e^{1.5} = 0.17$
$h_t(x_2) \neq c(x_2)$ #misclassified $\rightarrow D_2^{t+1} = D_2^t . e^{\alpha t} = 0.75.e^{1.5} = 3.36$

AdaBoost Inference: Kunapuli (2023)

```
def predict_boosting(X, estimators):
    pred = np.zeros((X.shape[0], ))

    for a, h in estimators:
        pred += a * h.predict(X)

    y = np.sign(pred)

    return y
```

Initializes all the predictions to 0

Makes weighted prediction for each example

Converts weighted predictions to –1/1 labels

$$H(x) = \sum_{t=1}^{T} \alpha_t h_t(x)$$

Base Model	Error(Mi)	ai
M1	e1	a1=0.5*ln[(1-e1)/e1]
M2	e2	a2=0.5*ln[(1-e2)/e2]
…	..	…
Mn	en	an=0.5*ln[(1-en)/en]

M1: + 1
M2: - 1
M3: + 1
M4: + 1
M5: - 1

pred=a1*1+a2*-1+a3*1+a4*1+a5*-1
Class: sign (pred)

Assumption: predict(X) return +1 or -1

Learning rate η in Adaboost

$$\alpha_t = \frac{1}{2}\log\left(\frac{1-\epsilon_t}{\epsilon_t}\right)$$

$$\alpha_j = \eta \, log \frac{1-r_j}{r_j}$$

Weight of jth predictor

Learning rate

- The original AdaBoost algorithm did not use learning rate as a hyperparameter and it has been added afterwards as an enhancement.
- Learning rate η controls the degree to which the weights of misclassified samples are increased (or boosted).
- Sklearn default η=1.0
- Kunapuli implement the weight using log with base natural (ln).

https://towardsdatascience.com/boosting-algorithms-in-machine-learning-part-i-adaboost-b9d86041a521/

AdaBoost vs Gradient Boosting

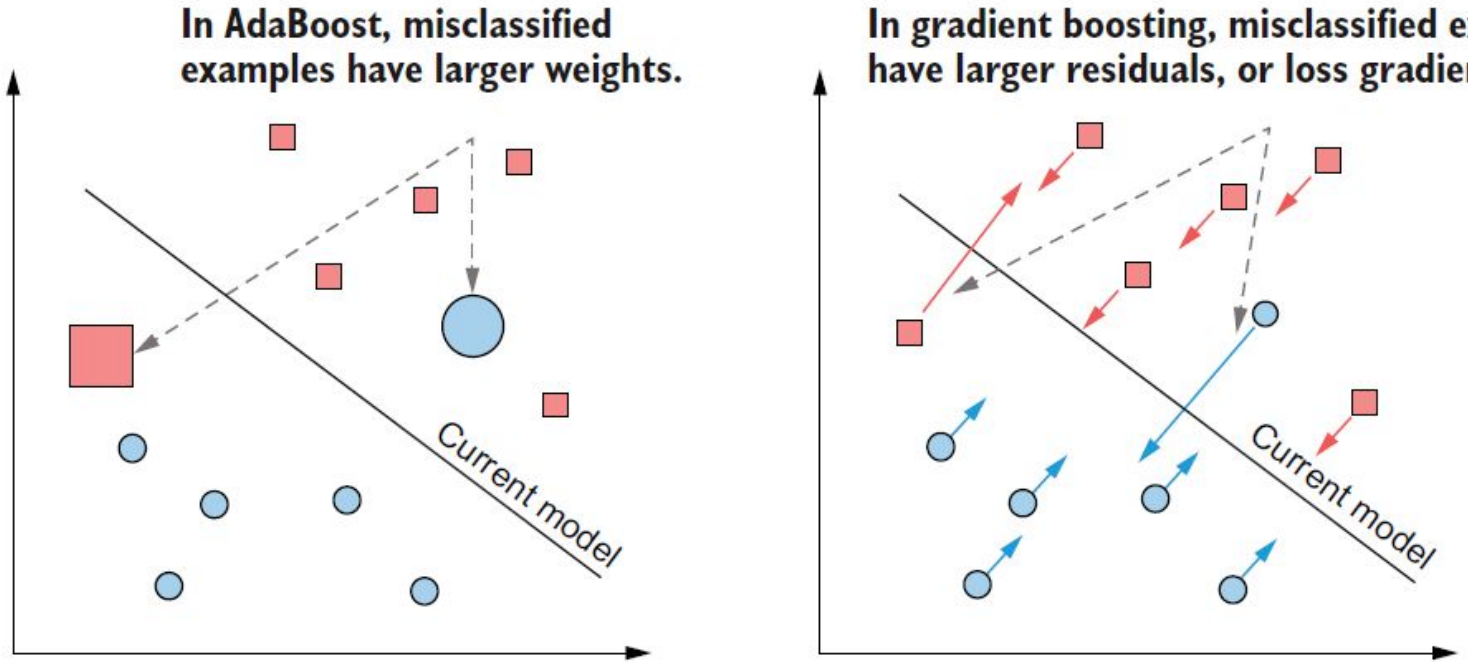


Figure 5.8 Comparing AdaBoost (left) to gradient boosting (right). Both approaches train weak estimators that improve classification performance on misclassified examples. AdaBoost uses weights, with misclassified examples being assigned higher weights. Gradient boosting uses residuals, with misclassified examples having higher residuals. The residuals are nothing but negative loss gradients.

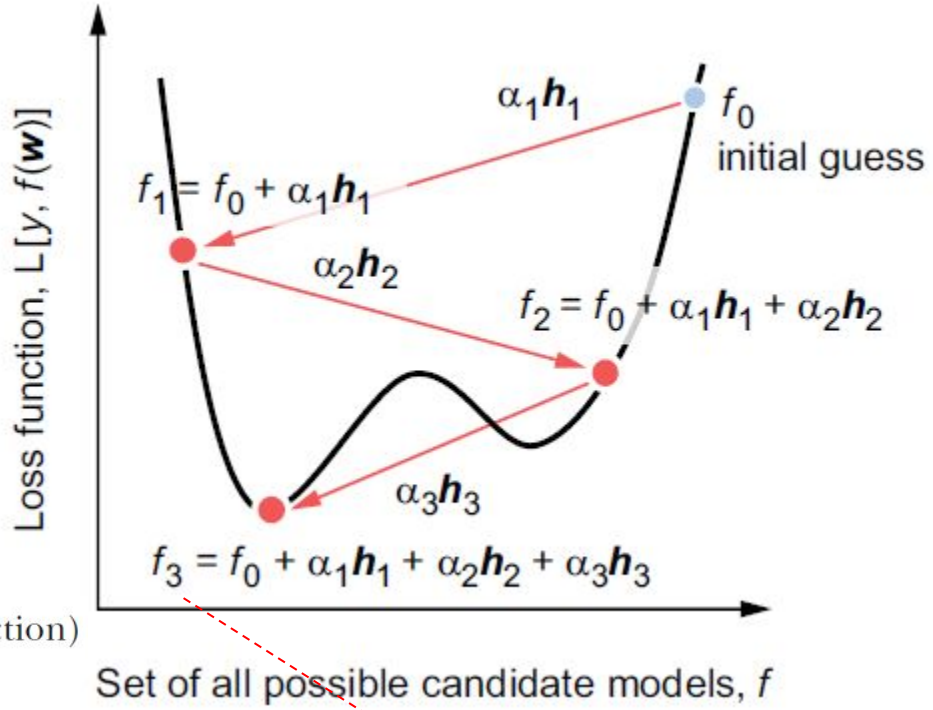
Gradient Boosting
=
Gradient Descent + Boosting

gradient boosting updates the current model with gradient information

gradient boosting trains a weak learner to fix the mistakes made by the previous weak learner.

Gradient Boosting

Each iteration additively updates the current model with the new approximate weak gradient estimate (the regression tree, ht).



new model = old model + (step length) * (update direction)

$$F_{t+1}(x) = F_t(x) + \alpha_t \cdot h_t(x)$$

$$F_{t+1}(x) = F_0(x) + \alpha_1 \cdot h_1(x) + \alpha_2 \cdot h_2(x) + \cdots + \alpha_{t-1} \cdot h_{t-1}(x) + \alpha_t \cdot h_t(x)$$

$$H(x) = \sum_{t=1}^{T} \alpha_t h_t(x)$$

Gradient Boosting: Residual is Negative Gradient

- residual(y,ŷ)=residual(y,h(x))=y-h(x)
- Squared loss function (SSE) to model h:

  $f_{loss}(y,\hat{y})=f_{loss}(y,h(x))=(y-h(x))^2/2$ $L(y,\hat{y}) = L(y,h(x)) = \frac{1}{2}(y-h(x))^2$

  $gradient(y,\hat{y})=\partial f_{loss}(y,h(x))/\partial h =-(y-h(x))$ $\text{gradient}(y,\hat{y}) = \frac{\partial L(y,h(x))}{\partial h} = -(y-h(x))$

  ($\text{gradient}(\text{true, predicted}) = \frac{\partial f_{\text{loss}}}{\partial h}(y,h(x)) = -(y-h(x))$
- Residual is the negative gradient:

  $residual(y,\hat{y})=-\partial f_{loss}/\partial h=y-h(x)$ # for squared loss $\text{residual}(y,\hat{y}) = -\frac{\partial L}{\partial h} = y-h(x)$

Gradient Boosting vs Gradient Descent

- instead of using the negative gradient, we use an approximate gradient trained on the negative residuals
- instead of checking for convergence, the algorithm terminates after a finite, maximum number of iterations T.

Gradient Boosting: Basic Algorithm

initialize: $F = f_0$, some constant value

for t = 1 to T:

1. compute the negative residuals for each example, $r_i^t = -\frac{\partial L}{\partial F}(\boldsymbol{x}_i)$
2. fit a weak decision tree regressor $h_t(\boldsymbol{x})$ using the training set $(\boldsymbol{x}_i, r_i)_{i=1}^n$
3. compute the step length (α_t) using line search
4. update the model: $F_t = F + \alpha_t \cdot h_t(\boldsymbol{x})$

Gradient Boosting: Training

```
from scipy.optimize import minimize_scalar
from sklearn.tree import DecisionTreeRegressor

def fit_gradient_boosting(X, y, n_estimators=10):
    n_samples, n_features = X.shape
    n_estimators = 10
    estimators = []
    F = np.full((n_samples, ), 0.0)

    for t in range(n_estimators):
        residuals = y - F
        h = DecisionTreeRegressor(max_depth=1)
        h.fit(X, residuals)

        hreg = h.predict(X)
        loss = lambda a: np.linalg.norm(
                              y - (F + a * hreg))**2
        step = minimize_scalar(loss, method='golden')
        a = step.x

        F += a * hreg
        estimators.append((a, h))

    return estimators
```

Gets dimensions of the data set

Initializes an empty ensemble

Predicts the ensemble on the training set

Computes residuals as negative gradients of the squared loss

Fits weak regression tree (h_t) to the examples and residuals

Gets predictions of the weak learner, h_t

Sets up the line search problem

Finds the best step length using the golden section search

Updates the ensemble predictions

Updates the ensemble

Gradient Boosting: alpha value

```
from scipy.optimize import minimize_scalar
from sklearn.tree import DecisionTreeRegressor

def fit_gradient_boosting(X, y, n_estimators=10):
    n_samples, n_features = X.shape
    n_estimators = 10
    estimators = []
    F = np.full((n_samples, ), 0.0)

    for t in range(n_estimators):
        residuals = y - F
        h = DecisionTreeRegressor(max_depth=1)
        h.fit(X, residuals)

        hreg = h.predict(X)
        loss = lambda a: np.linalg.norm(
                              y - (F + a * hreg))**2
        step = minimize_scalar(loss, method='golden')
        a = step.x

        F += a * hreg
        estimators.append((a, h))

    return estimators
```

Gets dimensions of the data set

Initializes an empty ensemble

Predicts the ensemble on the training set

Computes residuals as negative gradients of the squared loss

Fits weak regression tree (h_t) to the examples and residuals

Gets predictions of the weak learner, h_t

Sets up the line search problem

Finds the best step length using the golden section search

Updates the ensemble predictions

Updates the ensemble

```
In [31]:
1 from scipy.optimize import minimize_scalar
2 def f(x):
3     return 2*(x**2)+4*x-6
4 result = minimize_scalar(f)
5 x_min = result.x
6 x_min #x=-b/2a=-4/4=-1

Out[31]: -0.9999999851900002
```

Gradient Boosting: Inference

```
def predict_gradient_boosting(X, estimators):
    pred = np.zeros((X.shape[0], ))

    for a, h in estimators:
        pred += a * h.predict(X)

    y = np.sign(pred)

    return y
```

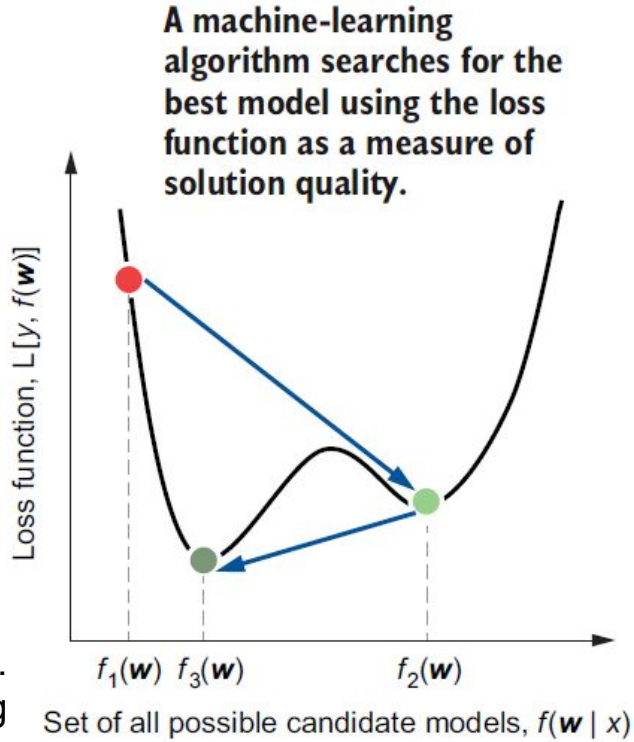
Initializes all the predictions to 0

Aggregates individual predictions from each regressor

Converts weighted predictions to –1/1 labels

Loss Function: Additional Slides

- Loss functions explicitly measure the fit of a model on a data set. Most often, we measure loss with respect to the true labels, by quantifying the error between the predicted and true labels.
- Given a loss function, training is the search for the optimal model that minimizes the loss, as illustrated in this figure.



An optimization procedure for finding the best model. Machine-learning algorithms search for the best model among all possible candidate models. The optimization procedure sequentially identifies increasingly better models f1, f2, and the final model, f3.

Gradient-based Optimization: Additional Slides

Many methods attempt to use the gradient of the landscape to find an optimum value. The gradient of the objective function is a vector ∇f that gives the magnitude and direction of the steepest slope.

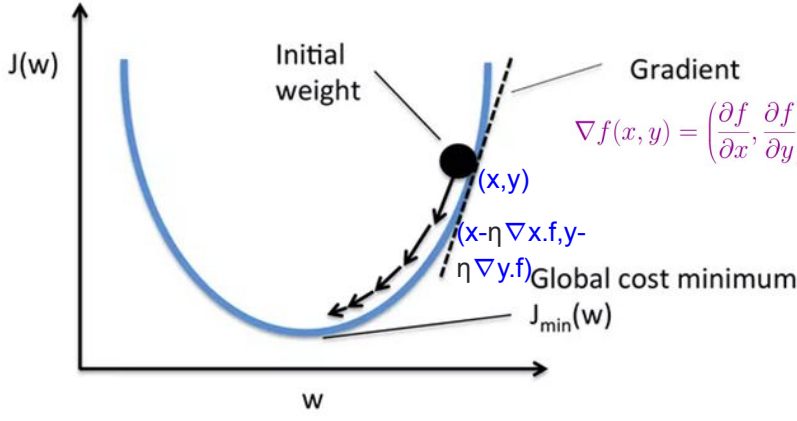
Given objective function f(x,y) to be minimized, and current state is (x,y), gradient is a vector:

$$\nabla f(x,y) = \left(\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}\right)$$

Use gradient to set successor state:

$x \leftarrow x - \eta \nabla x.f$

$y \leftarrow y - \eta \nabla y.f$



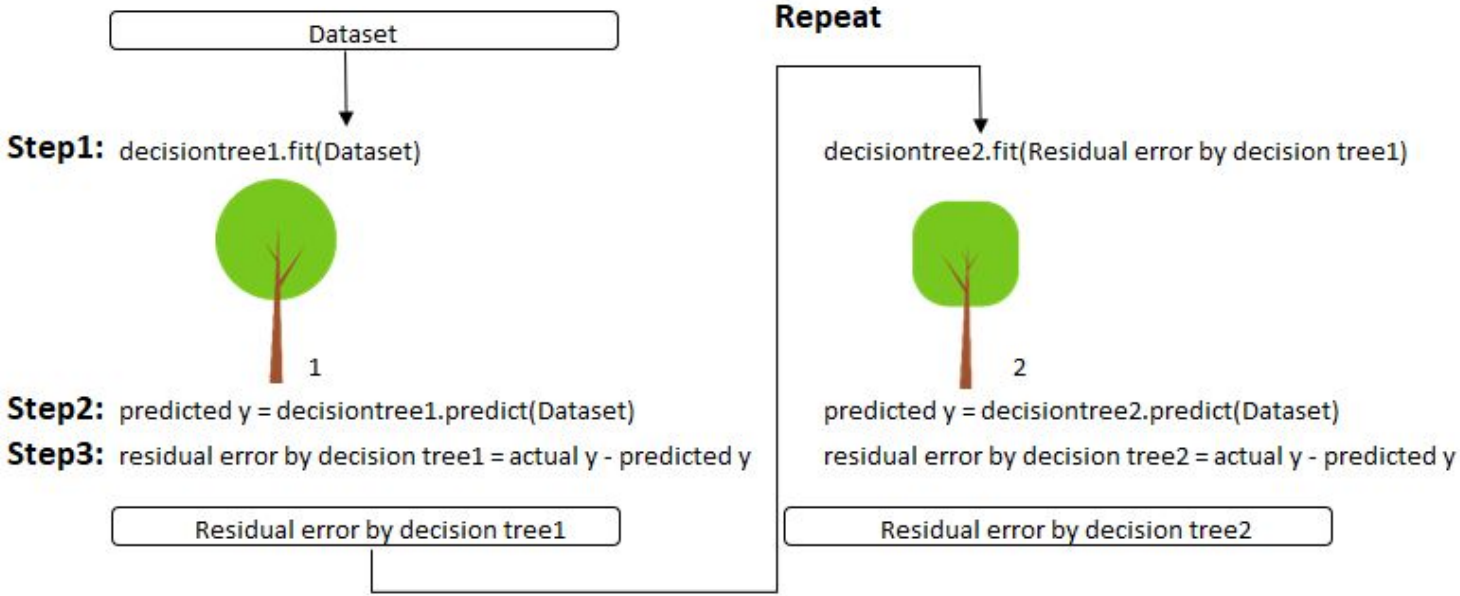
https://medium.com/data-science-group-iitr/loss-functions-and-optimization-algorithms-demystified-bb92daff331c

The basic problem is that if η is too small, too many steps are needed; if η is too large, the search could overshoot the maximum.

Eksplorasi Mandiri

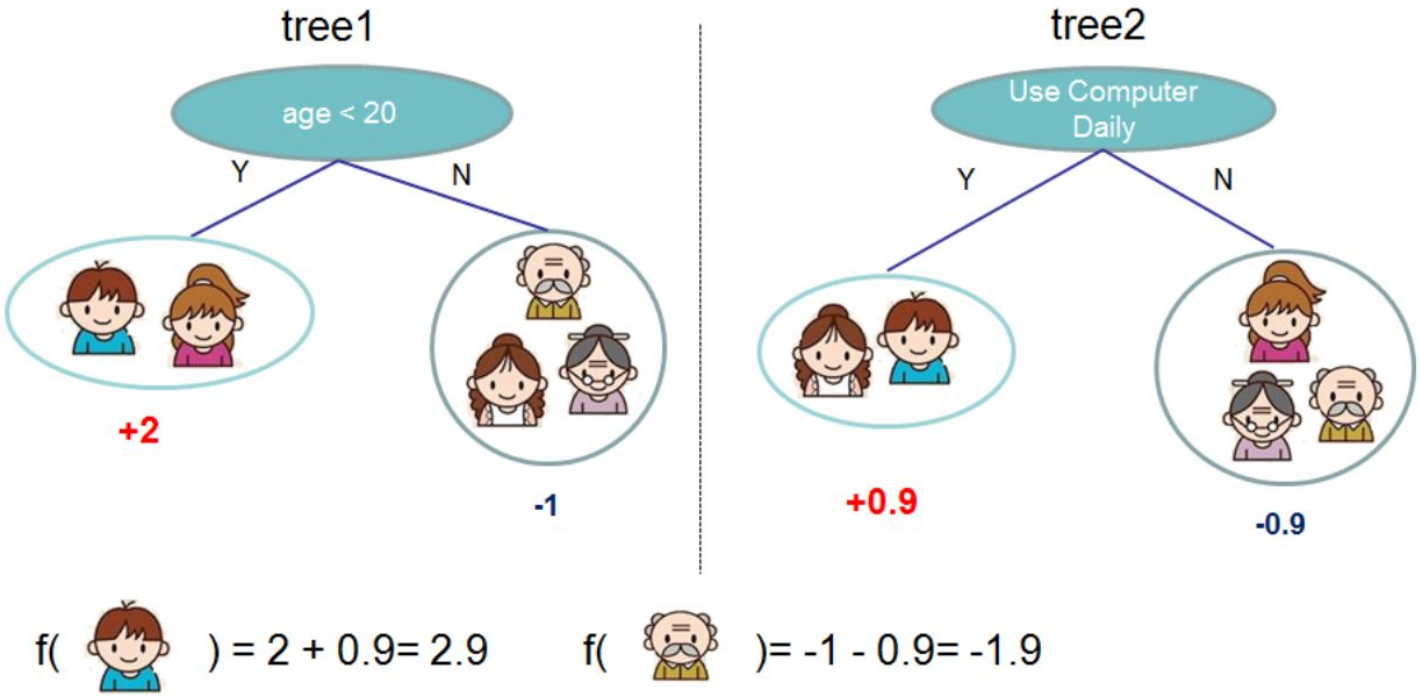
- Eksplorasi library `RandomForestClassifier`, `BaggingClassifier`, `AdaBoostClassifier`, `LightGBM` dari library `sklearn.ensemble`
- Ikuti hands-on pada subbab **2.5 Case study: Breast cancer diagnosis** buku teks Kunapuli, G. (2023). *Ensemble methods for machine learning*. Simon and Schuster

Gradient Boosting: Use Residual Error



https://towardsdatascience.com/basic-ensemble-learning-random-forest-adaboost-gradient-boosting-step-by-step-explained-95d49d1e2725

Gradient Boosting Prediction: Example



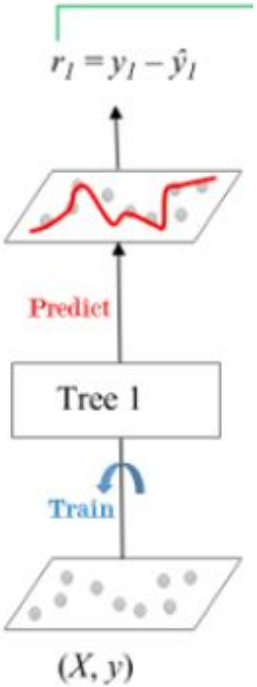
https://xgboost.readthedocs.io/en/latest/tutorials/model.html

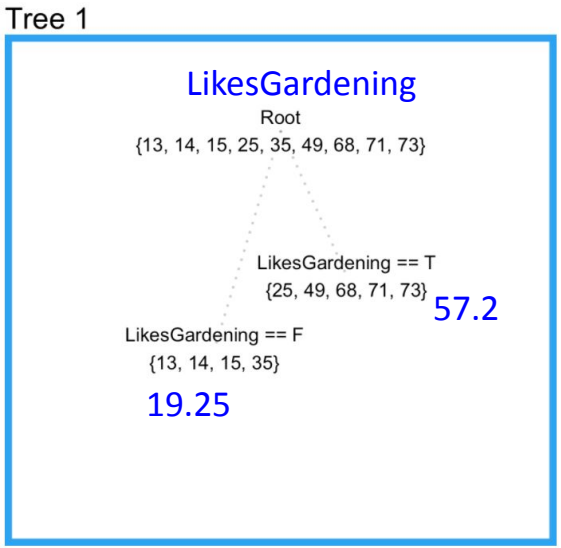
XGBoost Example: ***Predict Person's Age***

PersonID	Age	LikesGardening	PlaysVideoGames	LikesHats
1	13	FALSE	TRUE	TRUE
2	14	FALSE	TRUE	FALSE
3	15	FALSE	TRUE	FALSE
4	25	TRUE	TRUE	TRUE
5	35	FALSE	TRUE	TRUE
6	49	TRUE	FALSE	FALSE
7	68	TRUE	TRUE	TRUE
8	71	TRUE	FALSE	FALSE
9	73	TRUE	FALSE	TRUE

https://www.gormanalysis.com/blog/gradient-boosting-explained/#gradient-descent

XGBoost: Initialization $F_0(x)$=Tree1, Predict, & r1

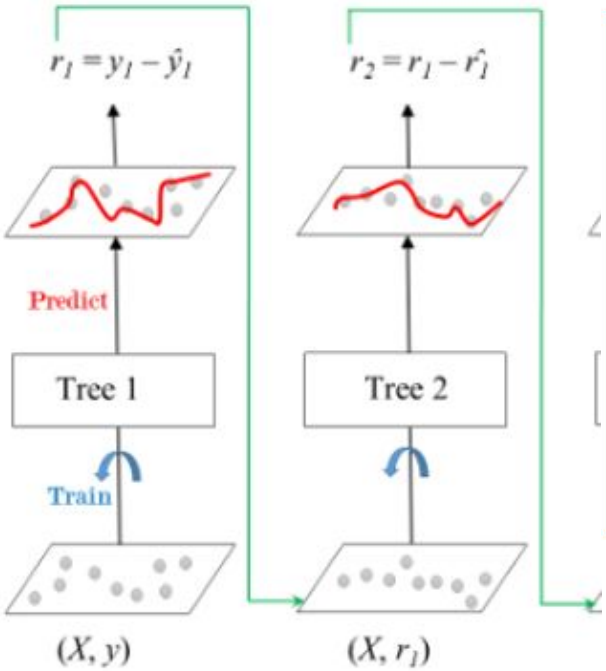




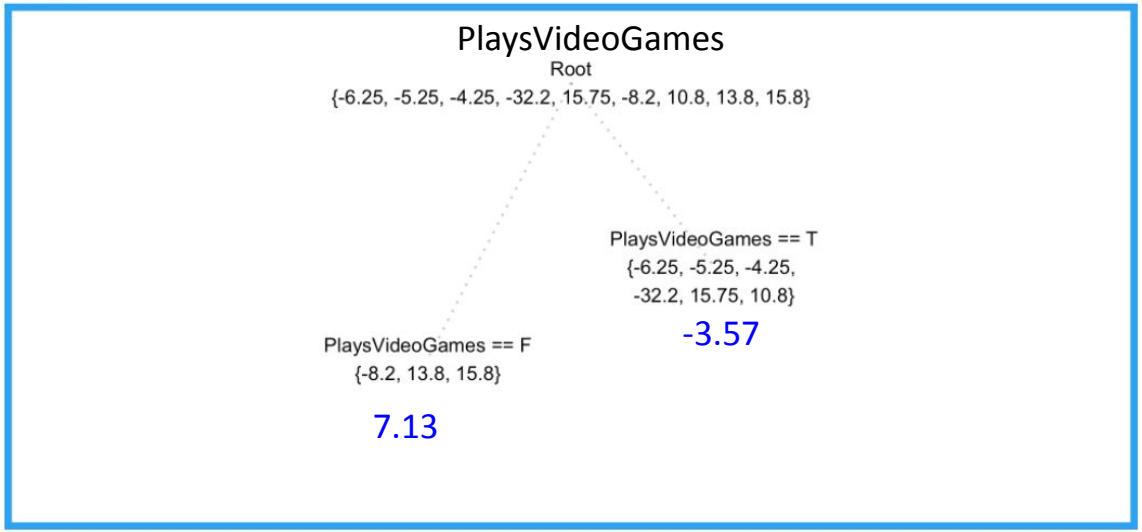
PersonID	Age	Tree1 Prediction	Tree1 Residual
1	13	19.25	-6.25
2	14	19.25	-5.25
3	15	19.25	-4.25
4	25	57.20	-32.20
5	35	19.25	15.75
6	49	57.20	-8.20
7	68	57.20	10.80
8	71	57.20	13.80
9	73	57.20	15.80

https://www.gormanalysis.com/blog/gradient-boosting-explained/#gradient-descent

XGBoost: Tree2 □ Combined Prediction

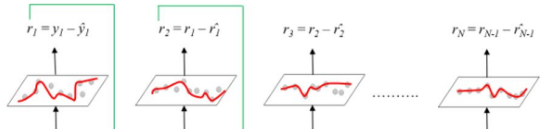


Tree2



https://www.gormanalysis.com/blog/gradient-boosting-explained/#gradient-descent

XGBoost:
$F_0(x)$=Tree1.predict+Tree2.predict



PersonID	Age	Tree1 Prediction	Tree1 Residual	Tree2 Prediction	Combined Prediction	Final Residual
1	13	19.25	-6.25	-3.57	15.68	2.68
2	14	19.25	-5.25	-3.57	15.68	1.68
3	15	19.25	-4.25	-3.57	15.68	0.68
4	25	57.20	-32.20	-3.57	53.63	28.63
5	35	19.25	15.75	-3.57	15.68	-19.32
6	49	57.20	-8.20	7.13	64.33	15.33
7	68	57.20	10.80	-3.57	53.63	-14.37
8	71	57.20	13.80	7.13	64.33	-6.67
9	73	57.20	15.80	7.13	64.33	-8.67

https://www.gormanalysis.com/blog/gradient-boosting-explained/#gradient-descent

Latihan di Kelas

Diberikan model ensemble dengan 3 base classifiers untuk klasifikasi biner (kelas Pos atau Neg). Setiap base classifier memberikan prediksi dalam bentuk peluang kelas Pos. Ketika diberikan sebuah instance **x**, hasil prediksinya dicantumkan di kolom Prediksi x. Lakukanlah inferensi model ensemble untuk mendapatkan hasil prediksi akhir apakah **x** termasuk dalam kelas Pos atau Neg dengan skema berikut. *Sebelum menjawab kelas prediksinya, berikanlah terlebih dahulu metode inferensi dari setiap skema*.

a. Bagging
b. Random Forest
c. Heterogeneous ensemble with weighting, voting=hard
d. Heterogeneous ensemble with meta-learning, dengan level 1 base-estimatorsnya adalah model 1 dan 2, dan level 2 estimator adalah model 3.
e. AdaBoost
f. Gradient Boosting. Bobot dari base classifier dalam bentuk α_i

Base-classifier	Akurasi validasi	Akurasi training
Model 1	0.5	0.60
Model 2	0.9	0.99
Model 3	0.6	0.90

Base-classifier	Prediksi **x**
Model 1	0.2
Model 2	0.75
Model 3	0.4

Questions ?